KUET_Phantoms Team Notebook

Sakiful Siam, Nihan Mahmud,
Mohammad Adib Chowdhury

Contents

1 Data Structure 1
1.1 2D Fenwick Tree [19 lines] - B5B7C5C9 1
1.2 Binary Trie (max XOR) [32 lines] - 02498358 . 2
1.3 Coordinate Compression [10 lines] - 964F1135 . 2
1.4 DSU with Rollback [23 lines] - B9BE1F42 . . . 2
1.5 DSU [12 lines] - CBA010E3 2
1.6 Dynamic Segment Tree [32 lines] - 5E509C15 . 2
1.7 Fast BIT and Iterative SegTree [56 lines] - EC40B789 2
1.8 Fenwick Tree [16 lines] - 0A127E9F 3
1.9 Lazy Segment Tree [33 lines] - 8852D1B5 3
1.10 Li Chao Tree (Dynamic CHT) [23 lines] - 8A23162A 3
1.11 MO with Updates [22 lines] - DEEB577D 3
1.12 MO [21 lines] - 365FA0CD 3
1.13 Monotonic Stack and Queue [49 lines] - D6C4C218 4
1.14 Persistent Segment Tree [26 lines] - AD80C226 4
1.15 Segment Tree [26 lines] - 83059A84 4
1.16 Sqrt Decomposition [16 lines] - 3BE3519B . . . 4
1.17 Sqrt Tricks [15 lines] - 88D92062 4
1.18 Static Range Queries [33 lines] - 5B167B1B . . 5
1.19 Treap (Implicit) [67 lines] - A3BFD86A 5
1.20 Wavelet Tree [21 lines] - 13646CA5 5

2 Dynamic Programming 5
2.1 Bitmask DP [14 lines] - C38E4F22 5
2.2 CHT DP [21 lines] - 404403AC 5
2.3 DP Fundamentals [11 lines] - 0794054D 6
2.4 DP with Fenwick Tree [16 lines] - 4EF7EF1F . . 6
2.5 Digit DP [15 lines] - 719EC5D4 6
2.6 Divide and Conquer DP [37 lines] - 3998A79F . 6
2.7 Expected Value DP [22 lines] - 52293ADF . . . 6
2.8 Knapsack and Common DP Patterns [67 lines] - 25FF2074 6
2.9 Knuth Optimization [19 lines] - E0D31F7B . . . 7
2.10 LIS and LCS [14 lines] - BE71A9A7 7
2.11 Matrix Exponentiation [20 lines] - 92FB502E . 7
2.12 SOS DP [11 lines] - 99BAFF2A 7

3 Flow 7
3.1 Dinic [64 lines] - A88163FB 7
3.2 Flow Applications and Reductions [31 lines] - D861B868 8
3.3 Hopcroft-Karp [44 lines] - 13DFB1D8 8
3.4 Hungarian Algorithm [42 lines] - 1F17BFEC . . 8
3.5 Kuhn [20 lines] - 3D9B43DE 9
3.6 Min-Cost Max-Flow [38 lines] - 65176019 9

4 Game Theory 9
4.1 Points to be noted [14 lines] - 2A4DE83E 9
4.2 Sprague-Grundy [31 lines] - 4A126A02 9

5 Geometry 9
5.1 Basic Geometry [32 lines] - 04827344 9
5.2 Closest Pair and Intersections [67 lines] - 6208C30F 10
5.3 Convex Hull [20 lines] - 1570DC7B 10
5.4 Half-Plane Intersection [58 lines] - AE2D7D69 . 10
5.5 Rotating Calipers [18 lines] - 4FFB9253 11

6 Graph 11
6.1 2-SAT [33 lines] - 817CF66C 11
6.2 Articulation Points and Bridges [26 lines] - 0579D87B 11
6.3 Bellman-Ford [17 lines] - 682245B2 11
6.4 Biconnected Components [36 lines] - 579687C0 11
6.5 Centroid Decomposition [41 lines] - CD6A1648 11
6.6 DSU on Tree [35 lines] - 2CA23FB4 12
6.7 Eulerian Path and Circuit [32 lines] - F1583A3D 12
6.8 Floyd-Warshall [12 lines] - 3A4CAC57 12
6.9 Graph Basics [71 lines] - 6CCDC4CA 12
6.10 Heavy Light Decomposition [18 lines] - A3AD48C0 13
6.11 Mo on Tree [52 lines] - EF2C0120 13
6.12 SCC [43 lines] - 498BB931 13
6.13 Tree DP [18 lines] - 3557D8FE 14
6.14 Tree Queries [28 lines] - 738F3996 14

7 Math 14
7.1 CRT [17 lines] - 6647AF89 14
7.2 Combinatorics and Probability [34 lines] - 64A3417C 14
7.3 Euler Phi and Mobius [24 lines] - 4089BFF8 . . 14
7.4 Euler Theorem and Power Towers [41 lines] - 90E18453 15
7.5 Exact GCD Counting [16 lines] - F3CAC16C . . 15
7.6 Floor Sum and BSGS [63 lines] - 1C09CEE5 . . 15
7.7 Gaussian Elimination [71 lines] - E05CFA68 . . 15
7.8 Linear Diophantine Equation [15 lines] - 8747D6A7 16
7.9 Lucas Theorem [21 lines] - 937ED62C 16
7.10 Miller-Rabin Pollard-Rho [43 lines] - 0FC59EF1 16
7.11 Modular Inverse [13 lines] - 6D6C77FA 16
7.12 NTT [53 lines] - 22422027 16
7.13 Sieve [42 lines] - B1AB1E33 17
7.14 Stirling Numbers [7 lines] - D3007645 17
7.15 Sum of Divisors [12 lines] - FCF9A6F7 17

8 Misc 17
8.1 Bit hacks [12 lines] - 1398A0E8 17
8.2 Contest Checklist and Complexity Table [33 lines] - 064C8435 17
8.3 STL Reference [28 lines] - 14293C11 18
8.4 Template [67 lines] - CA5EB085 18
8.5 XOR Hashing [15 lines] - 5F1277A0 18

9 Searching 18
9.1 Basic Array Patterns [38 lines] - 1DE99FBA . . 18
9.2 Basic Binary Search [27 lines] - 2333BA5B . . . 19
9.3 Range and Search Techniques [41 lines] - 8926C3DB 19

10 String 19
10.1 Aho-Corasick [27 lines] - A686271F 19
10.2 KMP [17 lines] - A1049B24 19
10.3 Lyndon and Hash61 [65 lines] - EC534088 . . . 19
10.4 Manacher [15 lines] - FBB15D9E 20
10.5 Palindromic Tree [48 lines] - 6F246768 20
10.6 Rolling Hash [30 lines] - B055754E 20
10.7 Suffix Array [29 lines] - 7C2108AD 20
10.8 Suffix Automaton [58 lines] - 9C63B26F 21
10.9 Trie [15 lines] - 48982540 21
10.10 Z-Algorithm [10 lines] - 7FEC0BF7 21

11 Basic Math and Formula Sheet 21
11.1 Integer Arithmetic 21
11.2 Sums and Counting 21
11.3 Modular Arithmetic 21
11.4 Bit and Geometry Reminders 21
11.5 Combinatorics 22
11.5.1 Catalan Number 22
11.5.2 Stirling Numbers (2nd Kind) 22
11.5.3 Lucas Theorem 22
11.5.4 Derangement 22
11.6 Number Theory 22
11.6.1 Mobius Function and Inversion 22
11.6.2 GCD and LCM 22
11.6.3 Pick's Theorem 22
11.6.4 Formula Cheatsheet 22

1 Data Structure

1.1 2D Fenwick Tree [19 lines] - B5B7C5C9

```
// Point add and rectangle sum on a 1-indexed grid.
    TIME: O(log n log m).
struct Fenwick2D {
    int n, m;
    vector<vector<long long>> bit;
    Fenwick2D(int n, int m) : n(n), m(m), bit(n+1,
        vector<long long>(m+1)) {}
    void add(int x, int y, long long value) {
        for (int i=x; i<=n; i+=i&-i)
            for (int j=y; j<=m; j+=j&-j) bit[i][j] +=
                value;
    }
    long long sumPrefix(int x, int y) const {
        long long ans=0;
        for (int i=x; i>0; i-=i&-i)
            for (int j=y; j>0; j-=j&-j) ans +=
                bit[i][j];
        return ans;
    }
    long long rectangleSum(int x1,int y1,int x2,int y2)
        const {
        return sumPrefix(x2,y2)-sumPrefix(x1-1,y2)-
            sumPrefix(x2,y1-1)+sumPrefix(x1-1,y1-1);
    }
};
```

1.2 Binary Trie (max XOR) [32 lines] - 02498358

```
// Binary trie for nonnegative 31-bit values; increase
    the top bit if constraints require it.
struct BinaryTrie {
    struct Node { int child[2] = {-1, -1}; };
    vector<Node> t{{}};
    void insert(int x) {
        int v = 0;
        for (int b = 30; b >= 0; --b) {
            int bit = (x >> b) & 1;
            if (t[v].child[bit] == -1) { t[v].child[bit]
                = t.size(); t.push_back({}); }
            v = t[v].child[bit];
        }
    }
    int maxXor(int x) const {
        int v = 0, result = 0;
        for (int b = 30; b >= 0; --b) {
            int bit = (x >> b) & 1, want = bit ^ 1;
            if (t[v].child[want] != -1) { result |= 1 <<
                b; v = t[v].child[want]; }
            else v = t[v].child[bit];
        }
        return result;
    }
};
// Max prefix-XOR versus suffix-XOR over every split
    (both sides nonempty).
int maxPrefixSuffixXor(const vector<int>& a) {
    int n=a.size(); if(n<2) return 0;
    vector<int> pref(n),suff(n); pref[0]=a[0];
    for(int i=1;i<n;++i) pref[i]=pref[i-1]^a[i];
    suff[n-1]=a[n-1]; for(int i=n-2;i>=0;--i)
        suff[i]=suff[i+1]^a[i];
    BinaryTrie trie; int answer=0;
    for(int split=0;split<n-1;++split) {
        trie.insert(pref[split]);
        answer=max(answer,trie.maxXor(suff[split+1])); }
    return answer;
}
```

1.3 Coordinate Compression [10 lines] - 964F1135

```
// WHEN: replace large ordered values with compact
    ranks. TIME: build O(n log n), lookup O(log n).
    ASSUMES: add all values before build; id is
    meaningful for values present in the list.
// Coordinate compression: preserves order while mapping
    values to [0..m-1].
template<class T>
struct Compressor {
    vector<T> values;
    void add(T x){values.push_back(x);}
    void build(){sort(values.begin(),values.end());
        values.erase(unique(values.begin(),
        values.end()),values.end());}
    int id(const T& x) const {return
        lower_bound(values.begin(),values.end(),x)-
        values.begin();}
    T value(int i) const {return values[i];}
};
```

1.4 DSU with Rollback [23 lines] - B9BE1F42

```
// WHEN: undo unions in offline dynamic connectivity.
    TIME: find/unite O(log n), rollback O(changes).
    ASSUMES: no path compression; snapshot is history
    size.
// No path compression: union by size keeps tree height
    O(log n), enabling rollback.
struct RollbackDSU {
    vector<int> parent,size;
    vector<pair<int,int>> history; // {child root, old
        size of receiving root}; {-1,-1} = no-op
    int components;
    RollbackDSU(int n):parent(n),size(n,1),
        components(n){iota(parent.begin(),parent.end(),
        0);}
    int find(int v) const { while(v!=parent[v])
        v=parent[v]; return v; }
    int snapshot() const { return history.size(); }
    bool unite(int a,int b) {
        a=find(a); b=find(b);
        if(a==b) { history.push_back({-1,-1}); return
            false; }
        if(size[a]<size[b]) swap(a,b);
        history.push_back({b,size[a]}); parent[b]=a;
            size[a]+=size[b]; --components; return
            true;
    }
    void rollback(int snap) {
        while((int)history.size()>snap) {
            auto [b,oldSize]=history.back();
                history.pop_back();
            if(b==-1) continue;
            int a=parent[b]; parent[b]=b;
                size[a]=oldSize; ++components;
        }
    }
};
```

1.5 DSU [12 lines] - CBA010E3

```
struct DSU {
    vector<int> parent, size;
    DSU(int n) : parent(n), size(n, 1) {
        iota(parent.begin(), parent.end(), 0); }
    int find(int v) { return parent[v] == v ? v :
        parent[v] = find(parent[v]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a; size[a] += size[b];
        return true;
    }
};
```

1.6 Dynamic Segment Tree [32 lines] - 5E509C15

```
// WHEN: sparse range add/sum over a huge domain. TIME:
    O(log C) per operation. ASSUMES: coordinates in
    [0,1e9]; sums fit long long.
// Dynamic range add / range sum on a large fixed domain
    [0, LIMIT].
struct DynamicSegmentTree {
    static const int LIMIT=1000000000;
    struct Node { long long sum=0,lazy=0; int
        left=-1,right=-1; };
    vector<Node> t{Node()};
    int makeNode(){t.push_back(Node());return
        (int)t.size()-1;}
    void apply(int p,int l,int r,long long
        x){t[p].sum+=(r-l+1LL)*x;t[p].lazy+=x;}
    void push(int p,int l,int r){
        if(l==r||t[p].lazy==0)return;
        int m=l+(r-l)/2;
        if(t[p].left==-1)t[p].left=makeNode();
        if(t[p].right==-1)t[p].right=makeNode();
        int a=t[p].left,b=t[p].right; long long
            z=t[p].lazy; t[p].lazy=0;
        apply(a,l,m,z); apply(b,m+1,r,z);
    }
    void add(int ql,int qr,long long x,int p,int l,int
        r){
        if(qr<l||r<ql)return;
        if(ql<=l&&r<=qr){apply(p,l,r,x);return;}
        push(p,l,r); int m=l+(r-l)/2;
        if(t[p].left==-1)t[p].left=makeNode();
        if(t[p].right==-1)t[p].right=makeNode();
        int a=t[p].left,b=t[p].right;
        add(ql,qr,x,a,l,m); add(ql,qr,x,b,m+1,r);
        t[p].sum=t[a].sum+t[b].sum;
    }
    long long sum(int ql,int qr,int p,int l,int r){
        if(qr<l||r<ql)return 0; if(ql<=l&&r<=qr)return
            t[p].sum;
        if(t[p].left==-1&&t[p].right==-1)return
            t[p].lazy*(min(r,qr)-max(l,ql)+1LL);
        push(p,l,r); int m=l+(r-l)/2; long long ans=0;
        if(t[p].left!=-1)ans+=sum(ql,qr,t[p].left,l,m);
        if(t[p].right!=-1)ans+=sum(ql,qr,t[p].right,m+
            1,r);
        return ans;
    }
    void add(int l,int r,long long
        x){add(l,r,x,0,0,LIMIT);}
    long long sum(int l,int r){return
        sum(l,r,0,0,LIMIT);}
};
```

1.7 Fast BIT and Iterative SegTree [56 lines] - EC40B789

```
// 1. Iterative Segment Tree (point update, range query
    on [l, r) in O(log n))
#include <bits/stdc++.h>
using namespace std;

template<typename T>
struct IterativeSegTree {
    int n; vector<T> tree;
    IterativeSegTree(int n) : n(n), tree(2 * n, 0) {}
    void update(int p, T val) {
        for (tree[p += n] = val; p > 1; p >>= 1) tree[p
            >> 1] = tree[p] + tree[p ^ 1];
    }
    T query(int l, int r) { // range [l, r)
        T res = 0;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res += tree[l++];
            if (r & 1) res += tree[--r];
        }
        return res;
    }
};

// 2. Fenwick Tree with Range Update and Range Query
    (1-indexed)
struct RangeFenwick {
    int n; vector<long long> b1, b2;
```

```
    RangeFenwick(int n) : n(n), b1(n + 1, 0), b2(n + 1,
        0) {}
    void add(vector<long long>& b, int idx, long long
        val) {
        for (; idx <= n; idx += idx & -idx) b[idx] +=
            val;
    }
    void rangeUpdate(int l, int r, long long val) {
        add(b1, l, val); add(b1, r + 1, -val);
        add(b2, l, val * (l - 1)); add(b2, r + 1, -val
            * r);
    }
    long long prefixSum(int idx) {
        long long sum1 = 0, sum2 = 0;
        for (int i = idx; i > 0; i -= i & -i) { sum1 +=
            b1[i]; sum2 += b2[i]; }
        return sum1 * idx - sum2;
    }
    long long rangeQuery(int l, int r) { return
        prefixSum(r) - prefixSum(l - 1); }
};

// 3. BIT Descent: find smallest index with prefix sum
    >= k in O(log n) (1-indexed frequencies >= 0)
struct BITDescent {
    int n; vector<int> tree;
    BITDescent(int n) : n(n), tree(n + 1, 0) {}
    void add(int idx, int val) { for (; idx <= n; idx
        += idx & -idx) tree[idx] += val; }
    int findKth(int k) { // returns 1-based index
        int idx = 0;
        for (int p = 1 << __lg(n); p > 0; p >>= 1) {
            if (idx + p <= n && tree[idx + p] < k) {
                idx += p;
                k -= tree[idx];
            }
        }
        return idx + 1;
    }
};
```

1.8 Fenwick Tree [16 lines] - 0A127E9F

```
// 1-indexed; point add, prefix/range sum in O(log n).
template<class T>
struct Fenwick {
    int n;
    vector<T> bit;
    Fenwick(int n = 0) : n(n), bit(n + 1, T{}) {}
    void add(int i, T value) {
        for (; i <= n; i += i & -i) bit[i] += value;
    }
    T sumPrefix(int i) const {
        T ans{};
        for (; i > 0; i -= i & -i) ans += bit[i];
        return ans;
    }
    T sumRange(int l, int r) const { return sumPrefix(r)
        - sumPrefix(l - 1); }
};
```

1.9 Lazy Segment Tree [33 lines] - 8852D1B5

```
// Range add and range sum; 0-indexed inclusive ranges.
struct LazySegmentTree {
    int n;
    vector<long long> tree, lazy;
    LazySegmentTree(int n) : n(n), tree(4 * n), lazy(4
        * n) {}
    void apply(int p, int l, int r, long long value) {
        tree[p] += (r - l + 1LL) * value;
        lazy[p] += value;
    }
    void push(int p, int l, int r) {
        if (lazy[p] == 0 || l == r) return;
        int m = (l + r) / 2;
        apply(p * 2, l, m, lazy[p]); apply(p * 2 + 1, m
            + 1, r, lazy[p]);
        lazy[p] = 0;
    }
    void add(int ql, int qr, long long value, int p, int
        l, int r) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) { apply(p, l, r,
            value); return; }
        push(p, l, r);
        int m = (l + r) / 2;
        add(ql, qr, value, p * 2, l, m); add(ql, qr,
            value, p * 2 + 1, m + 1, r);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }
    long long sum(int ql, int qr, int p, int l, int r) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[p];
        push(p, l, r);
        int m = (l + r) / 2;
        return sum(ql, qr, p * 2, l, m) + sum(ql, qr, p
            * 2 + 1, m + 1, r);
    }
    void add(int l, int r, long long value) { add(l, r,
        value, 1, 0, n - 1); }
    long long sum(int l, int r) { return sum(l, r, 1,
        0, n - 1); }
};
```

1.10 Li Chao Tree (Dynamic CHT) [23 lines] - 8A23162A

```
// WHEN: arbitrary line insertions and minimum queries.
    TIME: O(log C) per operation. ASSUMES: integer x in
    [-1e9,1e9]; products fit long long.
// Li Chao tree for minimum line queries on a fixed
    integer x-domain.
struct DynamicLiChao {
    static const int XMIN=-1000000000, XMAX=1000000000;
    struct Line { long long m=0,b=(1LL<<62); long long
        eval(int x) const { return m*x+b; } };
    struct Node { Line line; int left=-1,right=-1; };
    vector<Node> t{Node()};
    int makeNode(){t.push_back(Node());return
        (int)t.size()-1;}
    void add(Line nw,int p,int l,int r) {
        int m=l+(r-l)/2; bool
            lef=nw.eval(l)<t[p].line.eval(l),mid=
            nw.eval(m)<t[p].line.eval(m);
        if(mid)swap(nw,t[p].line); if(l==r)return;
        if(lef!=mid) {
            if(t[p].left==-1)t[p].left=makeNode(); int
            c=t[p].left; add(nw,c,l,m); }
        else { if(t[p].right==-1)t[p].right=makeNode();
            int c=t[p].right; add(nw,c,m+1,r); }
    }
    long long query(int x,int p,int l,int r) const {
        long long ans=t[p].line.eval(x); if(l==r)return
            ans; int m=l+(r-l)/2;
        int c=x<=m?t[p].left:t[p].right;
        if(c==-1)return ans;
        return min(ans,query(x,c,x<=m?l:m+1,x<=m?m:r));
    }
    void addLine(long long m,long long
        b){add({m,b},0,XMIN,XMAX);}
    long long query(int x) const {return
        query(x,0,XMIN,XMAX);}
};
```

1.11 MO with Updates [22 lines] - DEEB577D

```
// WHEN: offline range queries mixed with point
    assignments. TIME: about O((N+Q)N^(2/3)) moves.
    ASSUMES: 0-based inclusive ranges; each update
    records before/after values and each query its
    update-time.
// Offline range queries with point assignments. Example
    maintained answer: range sum.
struct MoUpdate { int pos, before, after; };
struct MoQuery3D { int l,r,time,id; };
vector<long long> moWithUpdates(vector<int> a,const
    vector<MoUpdate>& updates,vector<MoQuery3D> qs) {
    int n=a.size(),
        block=max(1,(int)pow(max(1,n),2.0/3));
    sort(qs.begin(),qs.end(),[&](auto x,auto y){
        int bx=x.l/block,by=y.l/block; if(bx!=by)return
            bx<by;
        int br=x.r/block,cr=y.r/block; if(br!=cr)return
            (bx&1)?br>cr:br<cr;
        return (br&1)?x.time>y.time:x.time<y.time;
    });
    vector<long long> ans(qs.size()); int L=0,R=-1,T=0;
        long long cur=0;
    auto add=[&](int i){cur+=a[i];}; auto remove=[&](int
        i){cur-=a[i];};
    auto apply=[&](int idx,bool forward){auto
        u=updates[idx]; int
        value=forward?u.after:u.before;
        if(L<=u.pos&&u.pos<=R) remove(u.pos);
            a[u.pos]=value; if(L<=u.pos&&u.pos<=R)
            add(u.pos);};
    for(auto q:qs) {
        while(T<q.time) apply(T++,true); while(T>q.time)
            apply(--T,false);
        while(L>q.l)add(--L); while(R<q.r)add(++R);
            while(L<q.l)remove(L++);
            while(R>q.r)remove(R--);
        ans[q.id]=cur;
    }
    return ans;
}
```

1.12 MO [21 lines] - 365FA0CD

```
// Static range queries. Implement add/remove to
    maintain the desired answer.
struct Query { int l, r, id; };
vector<long long> moQueries(const vector<int>& a,
    vector<Query> qs) {
    int n = a.size(), block = max(1, (int)sqrt(n));
    sort(qs.begin(), qs.end(), [&](const Query& x, const
        Query& y) {
        int bx = x.l / block, by = y.l / block;
        return bx == by ? ((bx & 1) ? x.r > y.r : x.r <
            y.r) : bx < by;
    });
    vector<long long> ans(qs.size());
```

```
    int L = 0, R = -1; long long cur = 0;
    auto add = [&](int i) { cur += a[i]; };
    auto remove = [&](int i) { cur -= a[i]; };
    for (auto q : qs) {
        while (L > q.l) add(--L);
        while (R < q.r) add(++R);
        while (L < q.l) remove(L++);
        while (R > q.r) remove(R--);
        ans[q.id] = cur;
    }
    return ans;
}
```

1.13 Monotonic Stack and Queue [49 lines] - D6C4C218

```
// Monotonic Stack & Queue: O(n) next greater, histogram
    largest rectangle, sliding window min/max
#include <bits/stdc++.h>
using namespace std;

// 1. Next Greater Element: returns index of next
    strictly greater element (or n if none)
vector<int> nextGreater(const vector<int>& a) {
    int n = a.size();
    vector<int> nxt(n, n);
    vector<int> st; // stores indices with decreasing
        values
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] < a[i]) {
            nxt[st.back()] = i;
            st.pop_back();
        }
        st.push_back(i);
    }
    return nxt;
}

// 2. Largest Rectangle in Histogram: O(n)
long long largestRectangleArea(const vector<long long>&
    h) {
    int n = h.size();
    vector<int> st;
    long long maxArea = 0;
    for (int i = 0; i <= n; ++i) {
        long long cur = (i == n ? 0 : h[i]);
        while (!st.empty() && h[st.back()] >= cur) {
            long long height = h[st.back()];
            st.pop_back();
            int width = st.empty() ? i : i - st.back()
                - 1;
            maxArea = max(maxArea, height * width);
        }
        st.push_back(i);
    }
    return maxArea;
}

// 3. Sliding Window Maximum for window size k in O(n)
vector<int> slidingWindowMax(const vector<int>& a, int
    k) {
    deque<int> dq; // stores indices with decreasing
        values
    vector<int> ans;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (!dq.empty() && dq.front() <= i - k)
            dq.pop_front();
        while (!dq.empty() && a[dq.back()] <= a[i])
            dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) ans.push_back(a[dq.front()]);
    }
    return ans;
}
```

1.14 Persistent Segment Tree [26 lines] - AD80C226

```
// WHEN: retain prefix versions and answer range kth
    queries. TIME: O(log n) per insert/query. ASSUMES:
    nonempty input; values are coordinate-compressed.
// Persistent frequency tree over compressed values; kth
    smallest in a subarray.
struct PersistentSegmentTree {
    struct Node { int left=0,right=0,sum=0; };
    vector<Node> t{{}}; vector<int> root; int n;
    PersistentSegmentTree(const vector<int>& a) {
        vector<int> v=a; sort(v.begin(),v.end());
            v.erase(unique(v.begin(),v.end()),v.end());
            n=v.size();
        root.push_back(0);
        for(int x:a) root.push_back(insert(root.back(),
            0,n-1,lower_bound(v.begin(),v.end(),x)-
            v.begin()));
        values=move(v);
    }
    vector<int> values;
    int insert(int old,int l,int r,int pos) {
        int cur=t.size(); t.push_back(t[old]);
            ++t[cur].sum;
        if(l<r) { int m=(l+r)/2; if(pos<=m)t[cur].left=
            insert(t[old].left,l,m,pos); else
            t[cur].right=insert(t[old].right,m+1,r,
            pos); }
        return cur;
    }
    int kthIndex(int leftRoot,int rightRoot,int k,int
        l,int r) const {
        if(l==r)return l;
        int leftCount=t[t[rightRoot].left].sum-
            t[t[leftRoot].left].sum,m=(l+r)/2;
        return k<=leftCount?kthIndex(t[leftRoot].left,
            t[rightRoot].left,k,l,
            m):kthIndex(t[leftRoot].right,
            t[rightRoot].right,k-leftCount,m+1,r);
    }
    int kth(int l,int r,int k) const { // inclusive
        zero-based [l,r]
        return values[kthIndex(root[l],root[r+1],k,0,n-
            1)];
    }
};
```

1.15 Segment Tree [26 lines] - 83059A84

```
// 0-indexed inclusive ranges; point assignment and
    range sum in O(log n).
struct SegmentTree {
    int n;
    vector<long long> tree;
    SegmentTree(const vector<long long>& a) :
        n((int)a.size()), tree(4 * max(1,
        (int)a.size())) { build(1, 0, n - 1, a); }
    void build(int p, int l, int r, const vector<long
        long>& a) {
        if (l == r) { tree[p] = a[l]; return; }
        int m = (l + r) / 2;
        build(p * 2, l, m, a); build(p * 2 + 1, m + 1,
            r, a);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }
    void setPoint(int i, long long value, int p, int l,
        int r) {
        if (l == r) { tree[p] = value; return; }
        int m = (l + r) / 2;
        if (i <= m) setPoint(i, value, p * 2, l, m);
            else setPoint(i, value, p * 2 + 1, m + 1,
            r);
        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }
    long long query(int ql, int qr, int p, int l, int r)
        const {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[p];
        int m = (l + r) / 2;
        return query(ql, qr, p * 2, l, m) + query(ql,
            qr, p * 2 + 1, m + 1, r);
    }
    void setPoint(int i, long long value) { setPoint(i,
        value, 1, 0, n - 1); }
    long long query(int l, int r) const { return
        query(l, r, 1, 0, n - 1); }
};
```

1.16 Sqrt Decomposition [16 lines] - 3BE3519B

```
// Range sum with point updates, 0-indexed; O(sqrt(n))
    per operation.
struct SqrtSum {
    int n, blockSize;
    vector<long long> a, block;
    SqrtSum(vector<long long> values) :
        n(values.size()), blockSize(max(1,
        (int)sqrt(n))), a(move(values)), block((n +
        blockSize - 1) / blockSize) {
        for (int i = 0; i < n; ++i) block[i / blockSize]
            += a[i];
    }
    void setValue(int i, long long value) { block[i /
        blockSize] += value - a[i]; a[i] = value; }
    long long query(int l, int r) const {
        long long ans = 0;
        while (l <= r && l % blockSize) ans += a[l++];
        while (l + blockSize - 1 <= r) { ans += block[l
            / blockSize]; l += blockSize; }
        while (l <= r) ans += a[l++];
        return ans;
    }
};
```

1.17 Sqrt Tricks [15 lines] - 88D92062

```
// Enumerate divisors in pairs up to sqrt(n),
    O(sqrt(n)).
vector<long long> divisors(long long n) {
    vector<long long> small, large;
    for(long long d=1;d<=n/d;++d) if(n%d==0) {
        small.push_back(d);
        if(d!=n/d) large.push_back(n/d);
    }
    reverse(large.begin(),large.end());
    small.insert(small.end(),large.begin(),large.end());
    return small;
```

```
}
// Group equal floor(n/i) values: ranges with the same
    quotient can be processed together.
template<class F> void forEqualQuotients(long long n,F
    process) {
    for(long long l=1,r;l<=n;l=r+1) { long long q=n/l;
        r=n/q; process(l,r,q); }
}
```

1.18 Static Range Queries [33 lines] - 5B167B1B

```
// Sparse table: immutable array, O(1) idempotent
    queries (min/max/gcd).
struct SparseTable {
    vector<vector<int>> st;
    vector<int> lg;
    SparseTable(const vector<int>& a) {
        int n = a.size(); lg.resize(n + 1);
        for (int i = 2; i <= n; ++i) lg[i] = lg[i/2] +
            1;
        st.assign(lg[n] + 1, vector<int>(n)); st[0] = a;
        for (int k = 1; k < (int)st.size(); ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                st[k][i] = min(st[k-1][i], st[k-1][i +
                    (1 << (k-1))]);
    }
    int queryMin(int l, int r) const {
        int k = lg[r-l+1]; return min(st[k][l],
            st[k][r-(1<<k)+1]);
    }
};

// Merge sort tree: static range count of values <= x,
    O(log^2 n) per query.
struct MergeSortTree {
    int n; vector<vector<int>> t;
    MergeSortTree(const vector<int>& a) : n(a.size()),
        t(4 * max(1, (int)a.size())) {
        build(1,0,n-1,a); }
    void build(int p,int l,int r,const vector<int>& a) {
        if (l==r) { t[p]={a[l]}; return; }
        int m=(l+r)/2; build(p*2,l,m,a);
            build(p*2+1,m+1,r,a);
        merge(t[p*2].begin(),t[p*2].end(),t[p*2+
            1].begin(),t[p*2+1].end(),
            back_inserter(t[p]));
    }
    int countLE(int ql,int qr,int x,int p,int l,int r)
        const {
        if (qr<l || r<ql) return 0;
        if (ql<=l && r<=qr) return
            upper_bound(t[p].begin(),t[p].end(),x)-
            t[p].begin();
        int m=(l+r)/2; return countLE(ql,qr,x,p*2,l,m)+
            countLE(ql,qr,x,p*2+1,m+1,r);
    }
    int countLE(int l,int r,int x) const { return
        countLE(l,r,x,1,0,n-1); }
};
```

1.19 Treap (Implicit) [67 lines] - A3BFD86A

```
// Implicit Treap (Cartesian Tree on Array): O(log n)
    split, merge, insert, erase, range reverse.
#include <bits/stdc++.h>
using namespace std;

static mt19937 treap_rng(1337);

struct Node {
    int val, prio, sz = 1;
    bool rev = false;
    Node *l = nullptr, *r = nullptr;
    Node(int val) : val(val), prio(treap_rng()) {}
};

int sz(Node* t) { return t ? t->sz : 0; }
void pull(Node* t) { if (t) t->sz = 1 + sz(t->l) +
    sz(t->r); }
void push(Node* t) {
    if (t && t->rev) {
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= 1;
        if (t->r) t->r->rev ^= 1;
        t->rev = false;
    }
}

void split(Node* t, int k, Node*& l, Node*& r) { // l
    gets first k elements, r gets rest
    push(t);
    if (!t) { l = r = nullptr; return; }
    if (sz(t->l) >= k) {
        split(t->l, k, l, t->l);
        r = t;
    } else {
        split(t->r, k - sz(t->l) - 1, t->r, r);
        l = t;
    }
    pull(t);
}

void merge(Node*& t, Node* l, Node* r) {
    push(l); push(r);
    if (!l || !r) { t = l ? l : r; return; }
    if (l->prio > r->prio) {
        merge(l->r, l->r, r);
        t = l;
    } else {
        merge(r->l, l, r->l);
        t = r;
    }
    pull(t);
}

// Reverse range [ql, qr] (0-indexed)
void reverseRange(Node*& root, int ql, int qr) {
    Node *a, *b, *c;
    split(root, qr + 1, b, c);
    split(b, ql, a, b);
    if (b) b->rev ^= 1;
    merge(root, a, b);
    merge(root, root, c);
}

void inorder(Node* t, vector<int>& res) {
    if (!t) return;
    push(t);
    inorder(t->l, res);
    res.push_back(t->val);
    inorder(t->r, res);
}
```

1.20 Wavelet Tree [21 lines] - 13646CA5

```
// WHEN: static range kth queries. TIME: O(log sigma)
    per query. ASSUMES: values fit the constructor
    domain; array does not change.
// Static range kth-smallest. Build on values in
    [lo,hi]; query uses inclusive zero-based indices.
struct WaveletTree {
    int lo,hi; vector<int> leftCount;
        unique_ptr<WaveletTree> left,right;
    WaveletTree(const vector<int>& a,int x,int
        y):lo(x),hi(y) {
        leftCount.assign(a.size()+1,0);
            if(a.empty()||lo==hi)return;
        int mid=lo+(hi-lo)/2; vector<int> l,r;
            l.reserve(a.size()); r.reserve(a.size());
        for(int i=0;i<(int)a.size();++i) {
            bool goLeft=a[i]<=mid;
                leftCount[i+1]=leftCount[i]+goLeft;
            (goLeft?l:r).push_back(a[i]);
        }
        if(!l.empty())left=make_unique<WaveletTree>(l,
            lo,mid);
        if(!r.empty())right=make_unique<WaveletTree>(r,
            mid+1,hi);
    }
    int kth(int l,int r,int k) const { // k is
        1-indexed
        if(lo==hi)return lo;
        int inLeft=leftCount[r+1]-leftCount[l];
        if(k<=inLeft)return
            left->kth(leftCount[l],leftCount[r+1]-1,k);
        return right->kth(l-leftCount[l],r-leftCount[r+
            1],k-inLeft);
    }
};
```

2 Dynamic Programming

2.1 Bitmask DP [14 lines] - C38E4F22

```
// Assignment-style bitmask DP: n <= about 20. dp[mask]
    = best cost after choosing mask.
long long assignmentDP(const vector<vector<int>>& cost)
    {
    int n=cost.size(), N=1<<n; const long long
        INF=(1LL<<60);
    vector<long long> dp(N,INF); dp[0]=0;
    for(int mask=0;mask<N;++mask) {
        int row=__builtin_popcount((unsigned)mask);
        if(row==n || dp[mask]==INF) continue;
        for(int col=0;col<n;++col) if(!(mask>>col&1)) {
            int next=mask|(1<<col); dp[next]=
                min(dp[next],dp[mask]+cost[row][col]);
        }
    }
    return dp[N-1];
}
```

2.2 CHT DP [21 lines] - 404403AC

```
// WHEN: DP transitions are line queries with ordered
    slopes and x values. TIME: O(1) amortized per
    operation. ASSUMES: slopes decrease and query x
    increases.
// Monotone convex hull trick for maximum of m*x+b:
    slopes decreasing, x queries increasing.
```

```
struct MonotoneCHT {
    struct Line { long long m,b; long long at(long long
        x) const { return m*x+b; } };
    deque<Line> hull;
    bool redundant(Line a,Line b,Line c) {
        return (__int128)(b.b-a.b)*(b.m-c.m)>=
            (__int128)(c.b-b.b)*(a.m-b.m);
    }
    void add(long long m,long long b) {
        Line x{m,b}; if(!hull.empty()&&hull.back().m==m)
            {
            if(hull.back().b>=b)return; hull.pop_back();
        }
        while(hull.size()>=
            2&&redundant(hull[hull.size()-2],
            hull.back(),x))hull.pop_back();
        hull.push_back(x);
    }
    long long query(long long x) {
        while(hull.size()>=2&&hull[0].at(x)<=
            hull[1].at(x))hull.pop_front();
        return hull.front().at(x);
    }
};
// Example recurrence: dp[i] = max_j(m[j]*x[i]+b[j]);
    add each candidate line, then query x[i].
```

2.3 DP Fundamentals [11 lines] - 0794054D

```
// State design: define what dp[...] means, transitions,
    base cases, and evaluation order.
// 0/1 knapsack: iterate capacity downward so each item
    is used at most once.
long long knapsack01(const vector<int>& w, const
    vector<long long>& val, int cap) {
    vector<long long> dp(cap + 1);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int c = cap; c >= w[i]; --c) dp[c] =
            max(dp[c], dp[c-w[i]] + val[i]);
    return dp[cap];
}
// Real-valued DP: use double states and explicit
    epsilon only for comparisons if needed.
// State swapping / rolling array: retain only the
    previous row when transitions need it.
// Recursion-to-iteration: process states in dependency
    order; replace recursion stack with a loop.
```

2.4 DP with Fenwick Tree [16 lines] - 4EF7EF1F

```
// WHEN: DP needs prefix aggregates after coordinate
    compression. TIME: O(n log n) here. ASSUMES:
    strictly increasing subsequence; max-sum variant.
// Maximum-sum strictly increasing subsequence;
    coordinate compression + Fenwick max.
struct FenwickMax {
    int n; vector<long long> bit;
    FenwickMax(int n):n(n),bit(n+1,0){}
    void update(int i,long long x) { for(;i<=n;i+=i&-i)
        bit[i]=max(bit[i],x); }
    long long query(int i) const { long long ans=0;
        for(;i;i-=i&-i) ans=max(ans,bit[i]); return
        ans; }
};
long long maxIncreasingSubsequenceSum(const vector<int>&
    a) {
    if(a.empty()) return 0;
    vector<int> vals=a; sort(vals.begin(),vals.end());
        vals.erase(unique(vals.begin(),vals.end()),
        vals.end());
    FenwickMax fw(vals.size()); long long
        answer=LLONG_MIN;
    for(int x:a) { int r=lower_bound(vals.begin(),
        vals.end(),x)-vals.begin()+1;
        long long cur=max((long
            long)x,fw.query(r-1)+x); fw.update(r,cur);
            answer=max(answer,cur); }
    return answer;
}
```

2.5 Digit DP [15 lines] - 719EC5D4

```
// Count integers in [0, bound] whose digit sum is
    divisible by k.
long long countDigitSumDivisible(const string& bound,int
    k) {
    int n=bound.size();
    vector<vector<array<long long,2>>>
        memo(n+1,vector<array<long long,2>>(k));
    vector<vector<array<char,2>>>
        seen(n+1,vector<array<char,2>>(k));
    function<long long(int,int,int)> dfs=[&](int pos,int
        rem,int tight)->long long {
        if(pos==n) return rem==0;
        if(seen[pos][rem][tight]) return
            memo[pos][rem][tight];
        seen[pos][rem][tight]=1;
        int lim=tight?bound[pos]-'0':9; long long ans=0;
        for(int d=0;d<=lim;++d)
            ans+=dfs(pos+1,(rem+d)%k,tight&&(d==lim));
        return memo[pos][rem][tight]=ans;
    };
    return dfs(0,0,1);
}
```

2.6 Divide and Conquer DP [37 lines] - 3998A79F

```
// Divide and Conquer DP Optimization: reduces O(K *
    N^2) to O(K * N log N).
// Condition: opt(i, j) <= opt(i, j + 1) for dp[i][j] =
    min_{k < j} (dp[i - 1][k] + cost(k, j)).
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;
int N, K;
vector<long long> dp_prev, dp_cur;

// Problem-specific cost function; should satisfy
    quadrangle inequality
long long cost(int k, int j) {
    return 0; // replace with C(k, j)
}

void compute(int l, int r, int optl, int optr) {
    if (l > r) return;
    int mid = (l + r) / 2;
    pair<long long, int> best = {INF, -1};
    for (int k = optl; k <= min(mid, optr); ++k) {
        long long cur = dp_prev[k] + cost(k, mid);
        if (cur < best.first) best = {cur, k};
    }
    dp_cur[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);
}

void solveDCDP() {
    dp_prev.assign(N + 1, INF);
    dp_cur.assign(N + 1, INF);
    dp_prev[0] = 0;
    for (int i = 1; i <= K; ++i) {
        compute(1, N, 0, N);
        dp_prev = dp_cur;
    }
}
```

2.7 Expected Value DP [22 lines] - 52293ADF

```
// Example state equations: expected coin tosses until
    two consecutive heads.
// State 0: previous toss is not H; state 1: previous
    toss is H; state 2 is terminal.
// E0=1+(E0+E1)/2 and E1=1+E0/2, so E0=6 and E1=4.
pair<double,double> expectedTossesForHH(){return
    {6.0,4.0};}
// For a finite acyclic process, fill E[state] from
    terminal states backward using
// E[s] = immediate_cost + sum(P(s->t) * E[t]). For
    cyclic transitions solve equations.

// Gaussian elimination for cyclic Markov expectation
    equations.
vector<double> solveLinearSystem(vector<vector<double>>
    a) { // augmented n x (n+1)
    int n=a.size();
    for(int col=0,row=0;col<n&&row<n;++col) {
        int pivot=row; for(int
            i=row+1;i<n;++i)if(abs(a[i][col])>
            abs(a[pivot][col]))pivot=i;
        if(abs(a[pivot][col])<1e-12)continue;
        swap(a[pivot],a[row]); double div=a[row][col];
        for(int j=col;j<=n;++j)a[row][j]/=div;
        for(int i=0;i<n;++i)if(i!=row){double
            factor=a[i][col];for(int
            j=col;j<=n;++j)a[i][j]-=factor*a[row][j];}
        ++row;
    }
    vector<double> x(n); for(int
        i=0;i<n;++i)x[i]=a[i][n]; return x;
}
// Build one equation per nonterminal state: E[u] -
    sum(P(u->v)*E[v]) = 1.
// Include only nonterminal v terms; terminal states
    have expectation 0.
```

2.8 Knapsack and Common DP Patterns [67 lines] - 25FF2074

```
// Tree Knapsack, Bounded Knapsack (Binary Splitting),
    Bitset Subset-Sum, and Space-Optimized Edit
    Distance
#include <bits/stdc++.h>
using namespace std;

// 1. Tree Knapsack: O(N * W) or O(N^2). Careful bounds
    iterate min(sub[u], W)
int n, W;
vector<vector<int>> g;
vector<int> sz, val, wt;
```

```cpp
vector<vector<long long>> dp; // dp[u][w] = max value in subtree u with weight w
void treeKnapsack(int u, int p) {
    sz[u] = 1;
    dp[u].assign(W + 1, 0);
    if (wt[u] <= W) dp[u][wt[u]] = val[u];
    for (int v : g[u]) if (v != p) {
        treeKnapsack(v, u);
        vector<long long> next_dp = dp[u];
        for (int i = 0; i <= min(sz[u], W); ++i) {
            for (int j = 0; j <= min(sz[v], W - i); ++j)
                {
                next_dp[i + j] = max(next_dp[i + j],
                    dp[u][i] + dp[v][j]);
            }
        }
        sz[u] += sz[v];
        dp[u] = move(next_dp);
    }
}

// 2. Bounded Knapsack with Binary Splitting: item with count C split into 1, 2, 4, ..., R
long long boundedKnapsack(const vector<int>& weights,
    const vector<long long>& values,
                          const vector<int>& counts, int
                              capacity) {
    vector<long long> dp(capacity + 1, 0);
    for (size_t i = 0; i < weights.size(); ++i) {
        int c = counts[i], w = weights[i]; long long v
            = values[i];
        for (int k = 1; c > 0; k = min(2 * k, c)) {
            int cur_w = k * w; long long cur_v = k * v;
            for (int cap = capacity; cap >= cur_w;
                --cap)
                dp[cap] = max(dp[cap], dp[cap - cur_w]
                    + cur_v);
            c -= k;
        }
    }
    return dp[capacity];
}

// 3. Bitset Subset-Sum: O(N * W / 64) reachability
bool subsetSumBitset(const vector<int>& a, int target) {
    bitset<1000005> bs;
    bs[0] = 1;
    for (int x : a) bs |= (bs << x);
    return bs[target];
}

// 4. Space-Optimized Edit Distance in O(N * M) time and O(M) memory
int editDistance(const string& s, const string& t) {
    int n = s.size(), m = t.size();
    vector<int> dp(m + 1);
    iota(dp.begin(), dp.end(), 0);
    for (int i = 1; i <= n; ++i) {
        int prev = dp[0];
        dp[0] = i;
        for (int j = 1; j <= m; ++j) {
            int temp = dp[j];
            if (s[i - 1] == t[j - 1]) dp[j] = prev;
            else dp[j] = 1 + min({prev, dp[j], dp[j -
                1]});
            prev = temp;
        }
    }
    return dp[m];
}
```

2.9 Knuth Optimization [19 lines] - E0D31F7B

```cpp
// WHEN: interval DP satisfies Knuth monotonicity. TIME: O(n^2). ASSUMES: prove the quadrangle inequality/monotone-opt property for the recurrence.
// Interval DP optimization. Requires a valid quadrangle/monotone-opt condition.
// Replace cost(l,r) with the problem's interval cost; indices are inclusive.
template<class Cost>
vector<vector<long long>> knuthDP(int n,Cost cost) {
    const long long INF=(1LL<<62);
    vector<vector<long long>> dp(n,vector<long
        long>(n));
    vector<vector<int>> opt(n,vector<int>(n));
    for(int i=0;i<n;++i)opt[i][i]=i;
    for(int len=2;len<=n;++len) for(int
        l=0;l+len<=n;++l) {
        int r=l+len-1; dp[l][r]=INF;
        int from=max(l,opt[l][r-1]),to=min(r-1,opt[l+
            1][r]);
        for(int k=from;k<=to;++k) {
            long long cur=(k>=l+1?dp[l][k]:0)+(k+1<=
                r?dp[k+1][r]:0)+cost(l,r);
            if(cur<dp[l][r])dp[l][r]=cur,opt[l][r]=k;
        }
    }
    return dp;
}
```

2.10 LIS and LCS [14 lines] - BE71A9A7

```cpp
// LIS length in O(n log n), strictly increasing.
int lisLength(const vector<int>& a) {
    vector<int> tails;
    for (int x : a) { auto
        it=lower_bound(tails.begin(),tails.end(),x);
        if(it==tails.end()) tails.push_back(x); else
        *it=x; }
    return tails.size();
}
// LCS length in O(n*m); can reduce memory by swapping to keep the shorter string in columns.
int lcsLength(const string& a, const string& b) {
    vector<int> dp(b.size()+1);
    for(char x:a) { int diagonal=0; for(int
        j=1;j<=(int)b.size();++j) {
        int old=dp[j]; dp[j]=(x==b[j-1]?diagonal+
            1:max(dp[j],dp[j-1])); diagonal=old;
    }}
    return dp.back();
}
```

2.11 Matrix Exponentiation [20 lines] - 92FB502E

```cpp
// Fibonacci in O(log n); adapt matrix and multiplication for a linear recurrence.
using Matrix = vector<vector<long long>>;
const long long MOD = 1000000007;
Matrix multiply(const Matrix& a,const Matrix& b) {
    int n=a.size(); Matrix c(n,vector<long long>(n));
    for(int i=0;i<n;++i) for(int k=0;k<n;++k) for(int
        j=0;j<n;++j)
        c[i][j]=(c[i][j]+a[i][k]*b[k][j])%MOD;
    return c;
}
Matrix power(Matrix a,long long e) {
    int n=a.size(); Matrix r(n,vector<long long>(n));
    for(int i=0;i<n;++i) r[i][i]=1;
    for(;e;e>>=1,a=multiply(a,a)) if(e&1)
        r=multiply(r,a);
    return r;
}
long long fibonacci(long long n) {
    if(n==0) return 0;
    Matrix base={{1,1},{1,0}};
    return power(base,n-1)[0][0];
}
```

2.12 SOS DP [11 lines] - 99BAFF2A

```cpp
// WHEN: aggregate a function over all submasks/supermasks. TIME: O(k*2^k). ASSUMES: k is small enough for 2^k memory.
// Subset zeta transform: after this, f[mask] = sum of original f[sub] over sub subset of mask.
void subsetZeta(vector<long long>& f,int bits) {
    for(int b=0;b<bits;++b) for(int
        mask=0;mask<(1<<bits);++mask)
        if(mask>>b&1) f[mask]+=f[mask^(1<<b)];
}
// Superset zeta: f[mask] becomes sum of original f[sup] where sup contains mask.
void supersetZeta(vector<long long>& f,int bits) {
    for(int b=0;b<bits;++b) for(int
        mask=0;mask<(1<<bits);++mask)
        if(!(mask>>b&1)) f[mask]+=f[mask|(1<<b)];
}
```

3 Flow

3.1 Dinic [64 lines] - A88163FB

```cpp
// Dinic's Algorithm for Maximum Flow. Complexity: O(V^2 E), for unit networks O(E sqrt(V)).
// Nodes 0-indexed. Capacities and flow use long long.
using ll = long long;
struct Dinic {
    struct Edge {
        int u, v;
        ll cap, flow = 0;
        Edge(int u, int v, ll cap) : u(u), v(v),
            cap(cap) {}
    };
    int N;
    vector<Edge> edge;
    vector<vector<int>> adj;
    vector<int> d, pt;
    Dinic(int N) : N(N), adj(N), d(N), pt(N) {}
    void addEdge(int u, int v, ll cap) {
        if (u == v) return;
        edge.emplace_back(u, v, cap);
        adj[u].push_back(edge.size() - 1);
        edge.emplace_back(v, u, 0);
        adj[v].push_back(edge.size() - 1);
    }
    bool bfs(int s, int t) {
        queue<int> q({ s });
        fill(d.begin(), d.end(), N + 1);
        d[s] = 0;
```

```
        while (!q.empty()) {
            int u = q.front(); q.pop();
            if (u == t) break;
            for (int k : adj[u]) {
                Edge& e = edge[k];
                if (e.flow < e.cap && d[e.v] > d[u] + 1)
                    {
                    d[e.v] = d[u] + 1;
                    q.push(e.v);
                }
            }
        }
        return d[t] != N + 1;
    }
    ll dfs(int u, int T, ll flow = -1) {
        if (u == T || flow == 0) return flow;
        for (int& i = pt[u]; i < (int)adj[u].size();
            i++) {
            Edge& e = edge[adj[u][i]];
            Edge& oe = edge[adj[u][i] ^ 1];
            if (d[e.v] == d[u] + 1) {
                ll amt = e.cap - e.flow;
                if (flow != -1 && amt > flow) amt =
                    flow;
                if (ll pushed = dfs(e.v, T, amt)) {
                    e.flow += pushed;
                    oe.flow -= pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }
    ll maxFlow(int s, int t) {
        ll total = 0;
        while (bfs(s, t)) {
            fill(pt.begin(), pt.end(), 0);
            while (ll flow = dfs(s, t)) total += flow;
        }
        return total;
    }
};
```

3.2 Flow Applications and Reductions [31 lines] - D861B868

```
1. MINIMUM CUT RECONSTRUCTION:
After computing max flow from source S to sink T, run
    BFS from S on residual graph:
traverse only edges where edge.flow < edge.cap.
Let reachable set be A, unreachable set be B (with T in
    B).
The minimum cut consists of original edges from A to B.
    Total cut capacity equals max flow.

2. PROJECT SELECTION / MAXIMUM WEIGHT CLOSURE:
Subset of tasks/projects S such that if u in S and u
    requires v, then v in S. Maximize sum of weights.
- If weight w_i > 0: add edge S -> i with capacity w_i.
- If weight w_i < 0: add edge i -> T with capacity
    |w_i|.
- If dependency u requires v: add edge u -> v with
    capacity INF.
Max profit = (sum of all positive w_i) - MinCut(S, T).
Selected projects are all vertices in the S-side of the
    minimum cut.

3. KÖNIG'S THEOREM (BIPARTITE MATCHING REDUCTIONS):
In any bipartite graph:
- Maximum Matching = Minimum Vertex Cover.
- Maximum Independent Set = Total Vertices - Maximum
    Matching.
- Minimum Edge Cover = Total Vertices - Maximum Matching
    (for graphs without isolated vertices).
To construct Min Vertex Cover:
Start DFS/BFS from all unmatched vertices on Left side
    using alternating paths (unmatched Left->Right,
    matched Right->Left).
Min Vertex Cover = (Left vertices NOT visited) + (Right
    vertices visited).

4. DILWORTH'S THEOREM & DAG PATH COVERS:
- Minimum Path Cover in DAG (disjoint paths): split each
    vertex v into v_out and v_in.
  Add edge u_out -> v_in for each DAG edge (u, v).
  Min Path Cover = |V| - MaxBipartiteMatching.
- General Path Cover (paths may share vertices): compute
    transitive closure (Floyd-Warshall) first,
  then apply disjoint path cover.
- Dilworth's Theorem: in any poset (DAG), the size of
    the minimum chain cover (path cover)
  equals the size of the maximum antichain (subset of
      vertices with no reachability between any pair).
```

3.3 Hopcroft-Karp [44 lines] - 13DFB1D8

```
// Maximum bipartite matching. Left vertices [0,n),
    right vertices [0,m).
// TIME: O(E sqrt(V)).
struct HopcroftKarp {
    int n, m;
    vector<vector<int>> g;
    vector<int> leftMatch, rightMatch, dist;
    int shortest;
    HopcroftKarp(int n, int m) : n(n), m(m), g(n),
        leftMatch(n,-1), rightMatch(m,-1), dist(n) {}
    void addEdge(int u, int v) { g[u].push_back(v); }
    bool bfs() {
        queue<int> q;
        for (int u=0; u<n; ++u) {
            if (leftMatch[u] == -1) dist[u]=0,
                q.push(u);
            else dist[u]=-1;
        }
        shortest=INT_MAX;
        while (!q.empty()) {
            int u=q.front(); q.pop();
            if (dist[u]+1>shortest) continue;
            for (int v:g[u]) {
                int w=rightMatch[v];
                if (w==-1) shortest=dist[u]+1;
                else if (dist[w]==-1)
                    dist[w]=dist[u]+1, q.push(w);
            }
        }
        return shortest!=INT_MAX;
    }
    bool dfs(int u) {
        for (int v:g[u]) {
            int w=rightMatch[v];
            if ((w==-1 && dist[u]+1==shortest) ||
                (w!=-1 && dist[w]==dist[u]+1 && dfs(w)))
                    {
                leftMatch[u]=v; rightMatch[v]=u; return
                    true;
            }
        }
        dist[u]=-1; return false;
    }
    int maximumMatching() {
        int ans=0;
        while (bfs()) for (int u=0; u<n; ++u)
            if (leftMatch[u]==-1 && dfs(u)) ++ans;
        return ans;
    }
};
```

3.4 Hungarian Algorithm [42 lines] - 1F17BFEC

```
// Hungarian Algorithm: Minimum Weight Maximum Bipartite
    Matching on N x M matrix in O(N^2 * M).
// 1-indexed. Returns {min_cost, assignment} where
    assignment[i] is matched right vertex for left
    vertex i.
#include <bits/stdc++.h>
using namespace std;

const long long INF = 1e18;

pair<long long, vector<int>> hungarian(int n, int m,
    const vector<vector<long long>>& cost) {
    vector<long long> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);
    for (int i = 1; i <= n; ++i) {
        p[0] = i;
        int j0 = 0;
        vector<long long> minv(m + 1, INF);
        vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0], j1 = 0;
            long long delta = INF;
            for (int j = 1; j <= m; ++j) {
                if (!used[j]) {
                    long long cur = cost[i0][j] - u[i0]
                        - v[j];
                    if (cur < minv[j]) { minv[j] = cur;
                        way[j] = j0; }
                    if (minv[j] < delta) { delta =
                        minv[j]; j1 = j; }
                }
            }
            for (int j = 0; j <= m; ++j) {
                if (used[j]) { u[p[j]] += delta; v[j]
                    -= delta; }
                else minv[j] -= delta;
            }
            j0 = j1;
        } while (p[j0] != 0);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    vector<int> match(n + 1);
    for (int j = 1; j <= m; ++j) if (p[j] <= n)
        match[p[j]] = j;
    return {-v[0], match};
}
```

3.5 Kuhn [20 lines] - 3D9B43DE

```
// Kuhn's algorithm for maximum bipartite matching; left
    side vertices 0..n-1.
struct Kuhn {
    int n, m;
    vector<vector<int>> g;
    vector<int> matchRight, seen;
    int timer = 0;
    Kuhn(int n, int m) : n(n), m(m), g(n),
        matchRight(m, -1), seen(n) {}
    void addEdge(int left, int right) {
        g[left].push_back(right); }
    bool dfs(int v) {
        if (seen[v] == timer) return false;
        seen[v] = timer;
        for (int to : g[v]) if (matchRight[to] == -1 ||
            dfs(matchRight[to])) { matchRight[to] = v;
            return true; }
        return false;
    }
    int maximumMatching() {
        int ans = 0;
        for (int v = 0; v < n; ++v) { ++timer; ans +=
            dfs(v); }
        return ans;
    }
};
```

3.6 Min-Cost Max-Flow [38 lines] - 65176019

```
// Successive shortest augmenting paths with SPFA. TIME:
    O(F*V*E).
// Supports negative edge costs; assumes no reachable
    negative-cost cycle.
struct MinCostMaxFlow {
    struct Edge { int to, rev, cap; long long cost; };
    int n;
    vector<vector<Edge>> g;
    MinCostMaxFlow(int n) : n(n), g(n) {}
    void addEdge(int u,int v,int cap,long long cost) {
        Edge a{v,(int)g[v].size(),cap,cost},
            b{u,(int)g[u].size(),0,-cost};
        g[u].push_back(a); g[v].push_back(b);
    }
    pair<int,long long> run(int s,int t) {
        int flow=0; long long cost=0;
        const long long INF=(1LL<<60);
        while (true) {
            vector<long long> d(n,INF); vector<int>
                pv(n,-1), pe(n,-1);
            vector<char> inQ(n); queue<int> q; d[s]=0;
                q.push(s); inQ[s]=1;
            while (!q.empty()) {
                int v=q.front(); q.pop(); inQ[v]=0;
                for (int i=0;i<(int)g[v].size();++i) {
                    auto &e=g[v][i];
                    if (e.cap && d[e.to]>d[v]+e.cost) {
                        d[e.to]=d[v]+e.cost;
                            pv[e.to]=v; pe[e.to]=i;
                        if (!inQ[e.to]) q.push(e.to),
                            inQ[e.to]=1;
                    }
                }
            }
            if (d[t]==INF) break;
            int add=INT_MAX;
            for (int v=t;v!=s;v=pv[v])
                add=min(add,g[pv[v]][pe[v]].cap);
            for (int v=t;v!=s;v=pv[v]) {
                Edge &e=g[pv[v]][pe[v]]; e.cap-=add;
                    g[v][e.rev].cap+=add;
            }
            flow+=add; cost+=1LL*add*d[t];
        }
        return {flow,cost};
    }
};
```

4 Game Theory

4.1 Points to be noted [14 lines] - 2A4DE83E

```
>[First Write a Brute Force solution]
>Nim = all xor
>Misere Nim = Nim + corner case: if all piles are 1,
    reverse(nim)
>Bogus Nim = Nim
>Staircase Nim = Odd indexed pile Nim (Even indexed pile
    doesnt matter, as one player can give bogus moves to
    drop all even piles to ground)
>Sprague Grundy: [Every impartial game under the normal
    play convention is equivalent to a one-heap game of
    nim]
Every tree = one nim pile = tree root value; tree leaf
    value = 0; tree node value = mex of all child nodes.
[Careful: one tree node can become multiple new tree
    roots(multiple elements in one node), then the value
    of that node = xor of all those root values]
>Hackenbush(Given a rooted tree; cut an edge in one
    move; subtree under that edge gets removed; last
    player to cut wins):
 Colon:  //G(u) = (G(v1) + 1) ⊕ (G(v2) + 1) ⊕ ··· [v1, v2, ···
    are childs of u]
For multiple trees ans is their xor
>Hackenbush on graph (instead of tree given an rooted
    graph):
fusion: All edges in a cycle can be fused to get a tree
    structure; build a super node, connect some single
    nodes with that super node, number of single nodes
    is the number of edges in the cycle.
Sol: [Bridge component tree] mark all bridges, a group
    of edges that are not bridges, becomes one component
    and contributes number of edges to the hackenbush.
    (even number of edges contributes 0, odd number of
    edges contributes 1)
```

4.2 Sprague-Grundy [31 lines] - 4A126A02

```
// Sprague-Grundy Theorem: mex calculation, DAG games,
    and subtraction game.
// An impartial game under normal play is winning iff
    XOR sum of Grundy values != 0.
#include <bits/stdc++.h>
using namespace std;

// 1. Minimum Excluded Value (mex)
int getMex(const vector<int>& vals) {
    vector<bool> seen(vals.size() + 2, false);
    for (int x : vals) if (x < (int)seen.size()) seen[x]
        = true;
    for (int i = 0; ; ++i) if (!seen[i]) return i;
}

// 2. Subtraction Game: from state n, player can
    subtract any s in allowed moves S
vector<int> subtractionGameGrundy(int maxN, const
    vector<int>& S) {
    vector<int> G(maxN + 1, 0);
    for (int n = 1; n <= maxN; ++n) {
        vector<int> nextStates;
        for (int s : S) if (n >= s)
            nextStates.push_back(G[n - s]);
        G[n] = getMex(nextStates);
    }
    return G;
}

// 3. Generic DAG Game Grundy values (memoized)
int memoG[100005];
int getGrundy(int u, const vector<vector<int>>& adj) {
    if (memoG[u] != -1) return memoG[u];
    vector<int> transitions;
    for (int v : adj[u])
        transitions.push_back(getGrundy(v, adj));
    return memoG[u] = getMex(transitions);
}
```

5 Geometry

5.1 Basic Geometry [32 lines] - 04827344

```
// WHEN: orientation, segment intersection, and polygon
    area. TIME: O(1) per primitive, O(n) area.
// Shared 2D Point structure with long long
    coordinates.
struct Point {
    long long x, y;
    bool operator<(const Point& o) const { return
        tie(x, y) < tie(o.x, o.y); }
    bool operator==(const Point& o) const { return x ==
        o.x && y == o.y; }
    Point operator+(const Point& o) const { return {x +
        o.x, y + o.y}; }
    Point operator-(const Point& o) const { return {x -
        o.x, y - o.y}; }
};
long long cross(Point a, Point b) { return a.x * b.y -
    a.y * b.x; }
long long cross(Point a, Point b, Point c) { return
    cross(b - a, c - a); }
long long dot(Point a, Point b) { return a.x * b.x + a.y
    * b.y; }
long long dist2(Point a, Point b) { return dot(a - b, a
    - b); }
long long orient(Point a, Point b, Point c) { return
    cross(a, b, c); }

bool onSegment(Point a, Point b, Point p) {
    return orient(a, b, p) == 0 && min(a.x, b.x) <= p.x
        && p.x <= max(a.x, b.x)
        && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}
bool segmentsIntersect(Point a, Point b, Point c, Point
    d) {
    long long x = orient(a, b, c), y = orient(a, b, d),
        z = orient(c, d, a), w = orient(c, d, b);
    if (x == 0 && onSegment(a, b, c)) return true;
    if (y == 0 && onSegment(a, b, d)) return true;
    if (z == 0 && onSegment(c, d, a)) return true;
    if (w == 0 && onSegment(c, d, b)) return true;
    return (x < 0) != (y < 0) && (z < 0) != (w < 0);
}
```

```cpp
long long doubledPolygonArea(const vector<Point>& p) {
    long long area = 0; int n = p.size();
    for (int i = 0; i < n; ++i) area += cross(p[i], p[(i
        + 1) % n]);
    return llabs(area); // shoelace formula; actual area
        is area / 2.0
}
```

5.2 Closest Pair and Intersections [67 lines] - 6208C30F

```cpp
// Closest Pair of Points, Point in Polygon, and
    Line/Circle Intersection
#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-9;

// 1. Closest Pair of Points: O(n log n) sweep-line
struct Pt {
    long long x, y; int id;
    bool operator<(const Pt& o) const { return x != o.x
        ? x < o.x : y < o.y; }
};
long long dist2(Pt a, Pt b) { return (a.x - b.x) * (a.x
    - b.x) + (a.y - b.y) * (a.y - b.y); }

pair<Pt, Pt> closestPair(vector<Pt> pts) {
    sort(pts.begin(), pts.end());
    struct CmpY { bool operator()(const Pt& a, const Pt&
        b) const { return a.y != b.y ? a.y < b.y : a.x <
        b.x; } };
    set<Pt, CmpY> active;
    long long bestD2 = LLONG_MAX;
    pair<Pt, Pt> bestPair;
    int left = 0;
    for (const auto& p : pts) {
        while (left < (int)pts.size() && 1LL * (p.x -
            pts[left].x) * (p.x - pts[left].x) >=
            bestD2) {
            active.erase(pts[left++]);
        }
        long long d = ceil(sqrt(bestD2));
        auto it1 = active.lower_bound({p.x, p.y - d,
            -1});
        auto it2 = active.upper_bound({p.x, p.y + d,
            -1});
        for (auto it = it1; it != it2; ++it) {
            long long d2 = dist2(p, *it);
            if (d2 < bestD2) { bestD2 = d2; bestPair =
                {p, *it}; }
        }
        active.insert(p);
    }
    return bestPair;
}

// 2. Point in Polygon: -1: outside, 0: on boundary, 1:
    strictly inside
struct Point2D { long long x, y; };
long long cross(Point2D a, Point2D b, Point2D c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) *
        (c.x - a.x);
}
int pointInPolygon(Point2D p, const vector<Point2D>&
    poly) {
    int n = poly.size(), winding = 0;
    for (int i = 0; i < n; ++i) {
        Point2D a = poly[i], b = poly[(i + 1) % n];
        if (min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
            &&
            min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y)
                && cross(a, b, p) == 0) return 0;
        if (a.y <= p.y) {
            if (b.y > p.y && cross(a, b, p) > 0)
                winding++;
        } else {
            if (b.y <= p.y && cross(a, b, p) < 0)
                winding--;
        }
    }
    return winding != 0 ? 1 : -1;
}

// 3. Line and Circle Intersection: circle centered at
    (0, 0) with radius r, line Ax + By + C = 0
// For general circle (cx, cy), translate line by C' = C
    + A*cx + B*cy.
vector<pair<double, double>>
    lineCircleIntersection(double r, double A, double
    B, double C) {
    double dist0 = abs(C) / sqrt(A * A + B * B);
    if (dist0 > r + EPS) return {}; // No intersection
    double x0 = -A * C / (A * A + B * B), y0 = -B * C /
        (A * A + B * B);
    if (abs(dist0 - r) < EPS) return {{x0, y0}}; //
        Tangent
    double d = sqrt(max(0.0, r * r - dist0 * dist0));
    double mult = sqrt(d * d / (A * A + B * B));
    return {{x0 + B * mult, y0 - A * mult}, {x0 - B *
        mult, y0 + A * mult}};
}
```

5.3 Convex Hull [20 lines] - 1570DC7B

```cpp
// Monotonic chain convex hull, O(n log n). CCW order
    without collinear points.
// Uses shared Point struct and cross(a, b, c) from
    Basic Geometry.
vector<Point> convexHull(vector<Point> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if (p.size() <= 1) return p;
    vector<Point> h;
    for (auto& pt : p) {
        while (h.size() >= 2 && cross(h[h.size() - 2],
            h.back(), pt) <= 0) h.pop_back();
        h.push_back(pt);
    }
    int lower = h.size();
    for (int i = (int)p.size() - 2; i >= 0; --i) {
        auto& pt = p[i];
        while ((int)h.size() > lower && cross(h[h.size()
            - 2], h.back(), pt) <= 0) h.pop_back();
        h.push_back(pt);
    }
    h.pop_back();
    return h; // Returns counterclockwise hull vertices
}
```

5.4 Half-Plane Intersection [58 lines] - AE2D7D69

```cpp
// Half-Plane Intersection: O(n log n). Half-plane is
    the left side of directed line p -> p + v.
#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-9;

struct Vec {
    double x, y;
    Vec(double x = 0, double y = 0) : x(x), y(y) {}
    Vec operator+(const Vec& o) const { return {x +
        o.x, y + o.y}; }
    Vec operator-(const Vec& o) const { return {x -
        o.x, y - o.y}; }
    Vec operator*(double k) const { return {x * k, y *
        k}; }
    double cross(const Vec& o) const { return x * o.y -
        y * o.x; }
};

struct Line {
    Vec p, v; double ang;
    Line() {}
    Line(Vec p, Vec v) : p(p), v(v), ang(atan2(v.y,
        v.x)) {}
    bool operator<(const Line& o) const { return ang <
        o.ang; }
};

Vec lineIntersection(const Line& a, const Line& b) {
    Vec u = a.p - b.p;
    double t = b.v.cross(u) / a.v.cross(b.v);
    return a.p + a.v * t;
}

bool onLeft(const Line& l, const Vec& pt) {
    return l.v.cross(pt - l.p) > EPS;
}

// Returns convex polygon formed by intersection of
    half-planes (or empty if unbounded/empty)
vector<Vec> halfPlaneIntersection(vector<Line> lines) {
    // Add large bounding box if needed: [-INF, INF]
    sort(lines.begin(), lines.end());
    vector<Line> q(lines.size() + 5);
    int head = 0, tail = 0;
    q[0] = lines[0];

    for (int i = 1; i < (int)lines.size(); ++i) {
        while (head < tail && !onLeft(lines[i],
            lineIntersection(q[tail], q[tail - 1])))
            tail--;
        while (head < tail && !onLeft(lines[i],
            lineIntersection(q[head], q[head + 1])))
            head++;
        if (abs(lines[i].v.cross(q[tail].v)) < EPS) {
            if (onLeft(lines[i], q[tail].p)) q[tail] =
                lines[i];
        } else {
            q[++tail] = lines[i];
        }
    }
    while (head < tail && !onLeft(q[head],
        lineIntersection(q[tail], q[tail - 1])))
        tail--;
    while (head < tail && !onLeft(q[tail],
        lineIntersection(q[head], q[head + 1])))
        head++;
    if (tail - head <= 1) return {};
```

```
    vector<Vec> poly;
    for (int i = head; i < tail; ++i)
        poly.push_back(lineIntersection(q[i], q[i +
        1]));
    poly.push_back(lineIntersection(q[tail], q[head]));
    return poly;
}
```

5.5 Rotating Calipers [18 lines] - 4FFB9253

```
// Diameter (farthest pair) of a convex polygon in CCW
    order, O(n).
// Use the convexHull template first; squared distances
    avoid floating point.
long long dist2(Point a,Point b) {
    long long dx=a.x-b.x, dy=a.y-b.y; return
        dx*dx+dy*dy;
}
long long convexDiameter2(const vector<Point>& p) {
    int n=p.size();
    if (n<=1) return 0;
    if (n==2) return dist2(p[0],p[1]);
    auto area2=[&](int i,int j,int k){ return
        llabs(cross(p[i],p[j],p[k])); };
    long long best=0;
    for (int i=0,j=1;i<n;++i) {
        int ni=(i+1)%n;
        while (area2(i,ni,(j+1)%n)>area2(i,ni,j))
            j=(j+1)%n;
        best=max({best,dist2(p[i],p[j]),dist2(p[ni],
            p[j])});
    }
    return best;
}
```

6 Graph

6.1 2-SAT [33 lines] - 817CF66C

```
// Boolean clauses (x == xv) OR (y == yv). TIME:
    O(V+E).
struct TwoSAT {
    int n;
    vector<vector<int>> g, rg;
    vector<int> order, comp;
    vector<char> seen;
    TwoSAT(int n) : n(n), g(2*n), rg(2*n) {}
    int node(int x, bool value) { return 2*x + !value; }
    void implication(int a, int b) { g[a].push_back(b);
        rg[b].push_back(a); }
    void addOr(int x, bool xv, int y, bool yv) {
        int a = node(x, xv), b = node(y, yv);
        implication(a^1, b); implication(b^1, a);
    }
    void dfs1(int v) {
        seen[v] = 1;
        for (int to : g[v]) if (!seen[to]) dfs1(to);
        order.push_back(v);
    }
    void dfs2(int v, int c) { comp[v] = c; for (int to :
        rg[v]) if (comp[to] == -1) dfs2(to, c); }
    bool solve(vector<int>& value) {
        seen.assign(2*n, 0); order.clear();
        for (int v = 0; v < 2*n; ++v) if (!seen[v])
            dfs1(v);
        comp.assign(2*n, -1);
        for (int i = 2*n-1, c = 0; i >= 0; --i)
            if (comp[order[i]] == -1) dfs2(order[i],
                c++);
        value.resize(n);
        for (int x = 0; x < n; ++x) {
            if (comp[2*x] == comp[2*x+1]) return false;
            value[x] = comp[2*x] > comp[2*x+1];
        }
        return true;
    }
};
```

6.2 Articulation Points and Bridges [26 lines] - 0579D87B

```
// WHEN: find articulation vertices and bridges in an
    undirected graph. TIME: O(V+E). ASSUMES: edge IDs
    distinguish parallel edges.
// Undirected graph; handles parallel edges by tracking
    the parent edge ID.
struct LowLink {
    int n, timer = 0;
    vector<vector<pair<int,int>>> g;
    vector<int> tin, low;
    vector<char> articulation;
    vector<pair<int,int>> bridges;
    LowLink(int n) : n(n), g(n), tin(n, -1), low(n),
        articulation(n) {}
    void addEdge(int u, int v, int id) {
        g[u].push_back({v,id}); g[v].push_back({u,id});
        }
    void dfs(int v, int parentEdge = -1) {
        tin[v] = low[v] = timer++;
        int children = 0;
        for (auto [to,id] : g[v]) {
            if (id == parentEdge) continue;
            if (tin[to] != -1) low[v] = min(low[v],
                tin[to]);
            else {
                dfs(to, id); low[v] = min(low[v],
                    low[to]); ++children;
                if (low[to] > tin[v])
                    bridges.push_back({v,to});
                if (parentEdge != -1 && low[to] >=
                    tin[v]) articulation[v] = true;
            }
        }
        if (parentEdge == -1 && children > 1)
            articulation[v] = true;
    }
    void run() { for (int v = 0; v < n; ++v) if (tin[v]
        == -1) dfs(v); }
};
```

6.3 Bellman-Ford [17 lines] - 682245B2

```
// Single-source shortest paths with negative edges.
    TIME: O(VE).
// Returns false iff a negative cycle is reachable from
    source.
struct Edge { int u, v; long long w; };
bool bellmanFord(int n, int source, const vector<Edge>&
    edges,
                 vector<long long>& dist) {
    const long long INF = (1LL << 60);
    dist.assign(n, INF); dist[source] = 0;
    for (int it = 0; it < n - 1; ++it) {
        bool changed = false;
        for (auto [u, v, w] : edges) if (dist[u] < INF
            && dist[v] > dist[u] + w)
            dist[v] = dist[u] + w, changed = true;
        if (!changed) break;
    }
    for (auto [u, v, w] : edges)
        if (dist[u] < INF && dist[v] > dist[u] + w)
            return false;
    return true;
}
```

6.4 Biconnected Components [36 lines] - 579687C0

```
// Vertex-biconnected edge blocks in an undirected
    graph. TIME: O(V+E).
// Edge IDs handle parallel edges; assumes no
    self-loops.
struct Biconnected {
    int n, timer = 0;
    vector<vector<pair<int,int>>> g;
    vector<int> tin, low, edgeStack;
    vector<char> articulation;
    vector<vector<int>> blocks;
    Biconnected(int n) : n(n), g(n), tin(n, -1),
        low(n), articulation(n) {}
    void addEdge(int u, int v, int id) {
        g[u].push_back({v,id}); g[v].push_back({u,id});
        }
    void dfs(int v, int parentEdge = -1) {
        tin[v] = low[v] = timer++;
        int children = 0;
        for (auto [to, id] : g[v]) {
            if (id == parentEdge) continue;
            if (tin[to] == -1) {
                edgeStack.push_back(id); ++children;
                    dfs(to, id);
                low[v] = min(low[v], low[to]);
                if (low[to] >= tin[v]) {
                    if (parentEdge != -1 || children >
                        1) articulation[v] = 1;
                    vector<int> block;
                    while (!edgeStack.empty()) {
                        int e = edgeStack.back();
                            edgeStack.pop_back();
                        block.push_back(e);
                        if (e == id) break;
                    }
                    blocks.push_back(move(block));
                }
            } else if (tin[to] < tin[v]) {
                edgeStack.push_back(id);
                low[v] = min(low[v], tin[to]);
            }
        }
    }
    void build() { for (int v = 0; v < n; ++v) if
        (tin[v] == -1) dfs(v); }
};
```

6.5 Centroid Decomposition [41 lines] - CD6A1648

```
// Centroid Decomposition: O(n log n). Skeleton with
    worked example for path counting.
#include <bits/stdc++.h>
using namespace std;

struct CentroidDecomposition {
    int n; vector<vector<int>> g; vector<int> sub;
        vector<char> blocked;
```

```
    CentroidDecomposition(int n) : n(n), g(n), sub(n),
        blocked(n) {}
    void addEdge(int u, int v) { g[u].push_back(v);
        g[v].push_back(u); }
    int calcSize(int v, int p) {
        sub[v] = 1;
        for (int u : g[v]) if (u != p && !blocked[u])
            sub[v] += calcSize(u, v);
        return sub[v];
    }
    int findCentroid(int v, int p, int total) {
        for (int u : g[v]) if (u != p && !blocked[u] &&
            sub[u] > total / 2)
            return findCentroid(u, v, total);
        return v;
    }
    // Worked example: count pairs of nodes with
        distance == K
    long long ans = 0; int K; vector<int> cnt;
    void getDists(int v, int p, int d, vector<int>& cur)
        {
        if (d > K) return;
        cur.push_back(d);
        for (int u : g[v]) if (u != p && !blocked[u])
            getDists(u, v, d + 1, cur);
    }
    void process(int c) {
        vector<int> modified; cnt[0] = 1;
            modified.push_back(0);
        for (int u : g[c]) if (!blocked[u]) {
            vector<int> cur; getDists(u, c, 1, cur);
            for (int d : cur) if (K >= d) ans += cnt[K
                - d];
            for (int d : cur) { cnt[d]++;
                modified.push_back(d); }
        }
        for (int d : modified) cnt[d] = 0; // reset freq
            array
    }
    void build(int entry = 0) {
        int total = calcSize(entry, -1), c =
            findCentroid(entry, -1, total);
        process(c);
        blocked[c] = 1;
        for (int u : g[c]) if (!blocked[u]) build(u);
    }
};
```

6.6 DSU on Tree [35 lines] - 2CA23FB4

```
// DSU on Tree (Sack): O(n log n) subtree queries.
    Small-to-large subtree merging.
#include <bits/stdc++.h>
using namespace std;

// Worked example: count distinct colors in each
    vertex's subtree
struct DSUOnTree {
    int n;
    vector<vector<int>> g;
    vector<int> col, sub, heavy, ans, cnt;
    int distinct = 0;
    DSUOnTree(int n, const vector<int>& col)
        : n(n), g(n), col(col), sub(n), heavy(n, -1),
            ans(n), cnt(1000005, 0) {}
    void addEdge(int u, int v) { g[u].push_back(v);
        g[v].push_back(u); }
    void calcSize(int v, int p) {
        sub[v] = 1; int best = 0;
        for (int u : g[v]) if (u != p) {
            calcSize(u, v); sub[v] += sub[u];
            if (sub[u] > best) { best = sub[u]; heavy[v]
                = u; }
        }
    }
    void add(int v, int p, int delta) {
        if (delta == 1) { if (++cnt[col[v]] == 1)
            distinct++; }
        else { if (--cnt[col[v]] == 0) distinct--; }
        for (int u : g[v]) if (u != p) add(u, v, delta);
    }
    void dfs(int v, int p, bool keep) {
        for (int u : g[v]) if (u != p && u != heavy[v])
            dfs(u, v, false);
        if (heavy[v] != -1) dfs(heavy[v], v, true);
        for (int u : g[v]) if (u != p && u != heavy[v])
            add(u, v, +1);
        if (++cnt[col[v]] == 1) distinct++;
        ans[v] = distinct;
        if (!keep) add(v, p, -1);
    }
    void run(int root = 0) { calcSize(root, -1);
        dfs(root, -1, true); }
};
```

6.7 Eulerian Path and Circuit [32 lines] - F1583A3D

```
// Undirected multigraph: Hierholzer's algorithm,
    O(V+E).
// Returns a vertex sequence using every edge once, or
    {} if impossible.
vector<int> eulerTrail(int n, const
    vector<pair<int,int>>& edges) {
    if (n == 0) return {};
    int m = edges.size();
    vector<vector<pair<int,int>>> g(n);
    vector<int> degree(n);
    for (int id = 0; id < m; ++id) {
        auto [u, v] = edges[id];
        g[u].push_back({v, id}); g[v].push_back({u,
            id});
        ++degree[u]; ++degree[v];
    }
    vector<int> odd;
    for (int v = 0; v < n; ++v) if (degree[v] & 1)
        odd.push_back(v);
    if (odd.size() != 0 && odd.size() != 2) return {};
    int start = odd.empty() ? 0 : odd[0];
    if (odd.empty()) for (int v = 0; v < n; ++v) if
        (degree[v]) { start = v; break; }
    vector<char> used(m);
    vector<int> it(n), st{start}, path;
    while (!st.empty()) {
        int v = st.back();
        while (it[v] < (int)g[v].size() &&
            used[g[v][it[v]].second]) ++it[v];
        if (it[v] == (int)g[v].size())
            path.push_back(v), st.pop_back();
        else {
            auto [to, id] = g[v][it[v]++];
            if (!used[id]) used[id] = 1,
                st.push_back(to);
        }
    }
    if ((int)path.size() != m + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}
```

6.8 Floyd-Warshall [12 lines] - 3A4CAC57

```
// All-pairs shortest paths. TIME: O(n^3), SPACE:
    O(n^2).
// INF means unreachable. Handles negative edges, but
    not negative cycles.
using ll = long long;
const ll INF = (1LL << 60);
void floydWarshall(vector<vector<ll>>& d) {
    int n = (int)d.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i) if (d[i][k] < INF)
            for (int j = 0; j < n; ++j) if (d[k][j] <
                INF)
                d[i][j] = min(d[i][j], d[i][k] +
                    d[k][j]);
}
// A negative cycle exists if d[u][u] < 0 after the
    algorithm.
```

6.9 Graph Basics [71 lines] - 6CCDC4CA

```
// WHEN: foundational graph traversals/shortest
    paths/MST. TIME: BFS/DFS/toposort O(V+E), Dijkstra
    O((V+E)logV), Kruskal O(E log E). ASSUMES: Dijkstra
    weights nonnegative; 0-1 BFS weights only 0/1.
// Unweighted graph: BFS distances and parents in
    O(V+E).
vector<int> bfsDistances(const vector<vector<int>>& g,
    int source) {
    vector<int> dist(g.size(), -1), parent(g.size(),
        -1);
    queue<int> q; dist[source]=0; q.push(source);
    while(!q.empty()) {
        int v=q.front(); q.pop();
        for(int u:g[v]) if(dist[u]==-1) {
            dist[u]=dist[v]+1; parent[u]=v; q.push(u);
        }
    }
    return dist;
}
// Multi-source BFS: push all source nodes into queue
    with dist[s]=0 initially, then run standard BFS.
// Finds shortest unweighted distance to the nearest
    source in O(V+E).

// Iterative DFS order; avoids recursion depth issues on
    large graphs.
vector<int> dfsOrder(const vector<vector<int>>& g,int
    source) {
    vector<char> seen(g.size()); vector<int> order,
        st{source}; seen[source]=1;
    while(!st.empty()) {
        int v=st.back(); st.pop_back();
            order.push_back(v);
        for(int u:g[v]) if(!seen[u])
            seen[u]=1,st.push_back(u);
    }
    return order;
}

// Dijkstra: nonnegative edge weights, O((V+E) log V).
```

```
vector<long long> dijkstra(const
    vector<vector<pair<int,int>>>& g,int source) {
    const long long INF=(1LL<<62);
    vector<long long> dist(g.size(),INF);
        priority_queue<pair<long
        long,int>,vector<pair<long
        long,int>>,greater<pair<long long,int>>> pq;
    dist[source]=0; pq.push({0,source});
    while(!pq.empty()) {
        auto [d,v]=pq.top(); pq.pop(); if(d!=dist[v])
            continue;
        for(auto [u,w]:g[v]) if(dist[u]>d+w)
            dist[u]=d+w,pq.push({dist[u],u});
    }
    return dist;
}

// 0-1 BFS: every edge weight must be 0 or 1; O(V+E).
vector<int> zeroOneBFS(const
    vector<vector<pair<int,int>>>& g,int source) {
    const int INF=1e9; vector<int> dist(g.size(),INF);
        deque<int> dq;
    dist[source]=0; dq.push_front(source);
    while(!dq.empty()) {
        int v=dq.front(); dq.pop_front();
        for(auto [u,w]:g[v]) if(dist[u]>dist[v]+w) {
            dist[u]=dist[v]+w;
                if(w==0)dq.push_front(u);else
                dq.push_back(u);
        }
    }
    return dist;
}

// Topological order by Kahn's algorithm; empty result
    means a directed cycle.
vector<int> topologicalSort(const vector<vector<int>>&
    g) {
    vector<int> indeg(g.size()),order;
    for(auto& edges:g) for(int u:edges) ++indeg[u];
    queue<int> q; for(int
        v=0;v<(int)g.size();++v)if(!indeg[v])q.push(v);
    while(!q.empty()) { int
        v=q.front();q.pop();order.push_back(v);for(int
        u:g[v])if(--indeg[u]==0)q.push(u); }
    if(order.size()!=g.size())return {}; return order;
}

// Kruskal MST for an undirected graph. DSU is local to
    keep this snippet standalone.
struct MSTEdge { int u,v; long long w; };
pair<bool,long long> kruskalMST(int n,vector<MSTEdge>
    edges) {
    sort(edges.begin(),edges.end(),[](auto a,auto
        b){return a.w<b.w;});
    vector<int> p(n),sz(n,1); iota(p.begin(),p.end(),0);
    function<int(int)> find=[&](int v){return
        p[v]==v?v:p[v]=find(p[v]);};
    long long total=0; int used=0;
    for(auto e:edges) { int
        a=find(e.u),b=find(e.v);if(a==b)continue;
        if(sz[a]<sz[b])swap(a,b);p[b]=a;sz[a]+=sz[b];
            total+=e.w;++used; }
    return {used==n-1,total}; // false for disconnected
        graph (except n=1)
}
```

6.10 Heavy Light Decomposition [18 lines] - A3AD48C0

```
// WHEN: path queries/updates on a static tree. TIME:
    O(log^2 n) path query. ASSUMES: 0-based connected
    tree; this template supports point assignment and
    path sum.
// HLD for point assignment and path sum; vertices are
    0-indexed.
struct HLD {
    int n,timer=0; vector<vector<int>> g; vector<int>
        parent,depth,heavy,head,pos,sub;
    vector<long long> base,seg;
    HLD(int n):n(n),g(n),parent(n,-1),depth(n),heavy(n,
        -1),head(n),pos(n),sub(n),base(n),seg(4*n){}
    void addEdge(int u,int
        v){g[u].push_back(v);g[v].push_back(u);}
    int dfsSize(int v,int p){parent[v]=p;sub[v]=1;int
        best=0;
        for(int u:g[v])if(u!=p){depth[u]=depth[v]+1;int
            z=dfsSize(u,v);sub[v]+=z;if(z>best)best=z,
            heavy[v]=u;}return sub[v];}
    void decompose(int v,int
        h){head[v]=h;pos[v]=timer++;
        if(heavy[v]!=-1)decompose(heavy[v],h);
        for(int u:g[v])if(u!=parent[v]&&u!=
            heavy[v])decompose(u,u);}
    void build(int
        root=0){dfsSize(root,-1);decompose(root,root);}
    void segSet(int i,long long x,int p,int l,int
        r){if(l==r){seg[p]=x;return;}int
        m=(l+r)/2;if(i<=m)segSet(i,x,p*2,l,m);else
        segSet(i,x,p*2+1,m+1,r);seg[p]=seg[p*2]+seg[p*
        2+1];}
    long long segSum(int ql,int qr,int p,int l,int
        r)const{if(qr<l||r<ql)return
        0;if(ql<=l&&r<=qr)return seg[p];int
        m=(l+r)/2;return segSum(ql,qr,p*2,l,m)+
        segSum(ql,qr,p*2+1,m+1,r);}
    void setValue(int v,long long
        x){segSet(pos[v],x,1,0,n-1);}
    long long pathSum(int a,int b)const{long long
        ans=0;while(head[a]!=head[b]){if(depth[head[a]]
        <depth[head[b]])swap(a,b);ans+=
        segSum(pos[head[a]],pos[a],1,0,n-1);a=
        parent[head[a]];}if(depth[a]>depth[b])swap(a,
        b);return ans+segSum(pos[a],pos[b],1,0,n-1);}
};
```

6.11 Mo on Tree [52 lines] - EF2C0120

```
// Mo's Algorithm on Tree: O((n + q) sqrt(n)) path
    queries on doubled Euler tour.
#include <bits/stdc++.h>
using namespace std;

struct TreeMo {
    struct Query { int l, r, lca, id; };
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> first, last, euler;
    TreeMo(int n) : n(n), g(n), first(n), last(n) {
        euler.reserve(2 * n); }
    void addEdge(int u, int v) { g[u].push_back(v);
        g[v].push_back(u); }
    void dfs(int v, int p) {
        first[v] = euler.size(); euler.push_back(v);
        for (int u : g[v]) if (u != p) dfs(u, v);
        last[v] = euler.size(); euler.push_back(v);
    }
    void build(int root = 0) { dfs(root, -1); }
    // If w is LCA: if w == u, query is [first[u],
        first[u]]; else [last[u], first[v]] + w
    Query makeQuery(int u, int v, int w, int id) const {
        if (first[u] > first[v]) swap(u, v);
        if (w == u) return {first[u], first[v], -1, id};
        return {last[u], first[v], w, id};
    }
    // Worked example: count distinct values on path u
        -> v
    vector<int> solve(vector<Query> qs, const
        vector<int>& val) const {
        int B = max(1, (int)(euler.size() / max(1.0,
            sqrt(qs.size()))));
        sort(qs.begin(), qs.end(), [&](const Query& a,
            const Query& b) {
            int ba = a.l / B, bb = b.l / B;
            return ba != bb ? ba < bb : ((ba & 1) ? a.r
                < b.r : a.r > b.r);
        });
        vector<int> ans(qs.size());
        vector<int> freq(1000005, 0);
        vector<char> active(n, 0);
        int distinct = 0, L = 0, R = -1;
        auto toggle = [&](int v) {
            active[v] ^= 1;
            int c = val[v];
            if (active[v]) { if (++freq[c] == 1)
                distinct++; }
            else { if (--freq[c] == 0) distinct--; }
        };
        for (auto& q : qs) {
            while (L > q.l) toggle(euler[--L]);
            while (R < q.r) toggle(euler[++R]);
            while (L < q.l) toggle(euler[L++]);
            while (R > q.r) toggle(euler[R--]);
            if (q.lca != -1) toggle(q.lca);
            ans[q.id] = distinct;
            if (q.lca != -1) toggle(q.lca);
        }
        return ans;
    }
};
```

6.12 SCC [43 lines] - 498BB931

```
/*components: number of SCC.
sz: size of each SCC.
comp: component number of each node.
Create reverse graph.
Run find_scc() to find SCC.
Might need to create condensation graph by
    create_condensed().
Think about indeg/outdeg
for multiple test cases- clear
    adj/radj/comp/vis/sz/topo/condensed.*/
vector<int>adj[mx], radj[mx];

int comp[mx], vis[mx], sz[mx], components;
vector<int>topo;
void dfs(int u) {
```

```
  vis[u] = 1;
  for (int v : adj[u])
    if (!vis[v]) dfs(v);
  topo.push_back(u);
}
void dfs2(int u, int val) {
  comp[u] = val;
  sz[val]++;
  for (int v : radj[u])
    if (comp[v] == -1)
      dfs2(v, val);
}
void find_scc(int n) {
  memset(vis, 0, sizeof vis);
  memset(comp, -1, sizeof comp);
  for (int i = 1;i <= n;i++)
    if (!vis[i])
      dfs(i);
  reverse(topo.begin(), topo.end());
  for (int u : topo)
    if (comp[u] == -1)
      dfs2(u, ++components);
}
vector<int>condensed[mx];
void create_condensed(int n) {
  for (int i = 1;i <= n;i++)
    for (int v : adj[i])
      if (comp[i] != comp[v])
        condensed[comp[i]].push_back(comp[v]);
}
```

6.13 Tree DP [18 lines] - 3557D8FE

```
// WHEN: compute subtree sizes and rerooted distance
    sums. TIME: O(n). ASSUMES: connected tree, 0-based
    vertices; recursive DFS may need an iterative
    rewrite for extreme depth.
// Tree DP: subtree sizes/sums, then reroot to get
    distance sum from every vertex.
// O(n), tree vertices 0..n-1; sumDist[u] = sum of
    distances from u to all vertices.
struct RerootTreeDP {
    int n; vector<vector<int>> g; vector<int> sub;
        vector<long long> down,sumDist;
    RerootTreeDP(int
        n):n(n),g(n),sub(n),down(n),sumDist(n){}
    void addEdge(int u,int
        v){g[u].push_back(v);g[v].push_back(u);}
    void dfs1(int v,int p){
        sub[v]=1; down[v]=0;
        for(int u:g[v])if(u!=p){dfs1(u,v);sub[v]+=
            sub[u];down[v]+=down[u]+sub[u];}
    }
    void dfs2(int v,int p){
        for(int u:g[v])if(u!=p){sumDist[u]=sumDist[v]+
            n-2LL*sub[u];dfs2(u,v);}
    }
    vector<long long> run(int root=0){
        dfs1(root,-1);sumDist[root]=down[root];
            dfs2(root,-1);return sumDist;
    }
};
```

6.14 Tree Queries [28 lines] - 738F3996

```
// WHEN: LCA, ancestor checks, and subtree intervals.
    TIME: O(log n) per LCA/lift. ASSUMES: rooted tree
    with vertices 1..n; call dfs(root,0).
// Binary lifting LCA and Euler intervals for one rooted
    tree (vertices 1..n).
struct TreeQueries {
    int n, LOG, timer = 0;
    vector<vector<int>> g, up;
    vector<int> depth, tin, tout, euler, doubleEuler;
    TreeQueries(int n) : n(n), LOG(1), g(n+1),
        depth(n+1), tin(n+1), tout(n+1) {
        while ((1 << LOG) <= n) ++LOG;
        up.assign(LOG, vector<int>(n+1));
            euler.reserve(n);
    }
    void addEdge(int u, int v) { g[u].push_back(v);
        g[v].push_back(u); }
    void dfs(int v, int p) {
        tin[v] = timer++; euler.push_back(v);
            doubleEuler.push_back(v);
        up[0][v] = p;
        for (int j = 1; j < LOG; ++j) up[j][v] =
            up[j-1][up[j-1][v]];
        for (int to : g[v]) if (to != p) { depth[to] =
            depth[v] + 1; dfs(to, v); }
        tout[v] = timer; doubleEuler.push_back(v);
    }
    int lift(int v, int k) const { for (int j = 0; j <
        LOG; ++j) if (k >> j & 1) v = up[j][v]; return
        v; }
    int lca(int a, int b) const {
        if (depth[a] < depth[b]) swap(a, b);
        a = lift(a, depth[a] - depth[b]);
        if (a == b) return a;
        for (int j = LOG - 1; j >= 0; --j) if (up[j][a]
            != up[j][b]) a = up[j][a], b = up[j][b];
        return up[0][a];
    }
    bool isAncestor(int a, int b) const { return tin[a]
        <= tin[b] && tout[b] <= tout[a]; }
};
```

7 Math

7.1 CRT [17 lines] - 6647AF89

```
// Merge x=a (mod m), x=b (mod n), including non-coprime
    moduli.
long long extendedGcdCRT(long long a,long long b,long
    long& x,long long& y) {
    if(!b) { x=1; y=0; return a; }
    long long x1,y1,g=extendedGcdCRT(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g;
}
// Returns {residue, modulus}; modulus 0 means no
    solution. Intermediate products must fit.
pair<long long,long long> mergeCRT(long long a,long long
    m,long long b,long long n) {
    long long x,y,g=extendedGcdCRT(m,n,x,y), diff=b-a;
    if(diff%g) return {0,0};
    long long mod=n/g;
    __int128 k=(__int128)(diff/g)*x%mod;
    long long lcm=m*mod;
    long long r=(long
        long)(((__int128)a+(__int128)m*k)%lcm);
    if(r<0) r+=lcm;
    return {r,lcm};
}
```

7.2 Combinatorics and Probability [34 lines] - 64A3417C

```
// Formula reference: see the PDF Random section (10)
    for Catalan, Stirling, derangements, and
    number-theory identities.
// Factorials and nCr modulo prime MOD. Requires n <
    MOD.
const long long MOD = 1000000007;
long long modPow(long long a, long long e) {
    long long r = 1;
    for (; e; e >>= 1, a = a * a % MOD) if (e & 1) r = r
        * a % MOD;
    return r;
}
struct Combinatorics {
    vector<long long> fact, invFact;
    Combinatorics(int n) : fact(n + 1), invFact(n + 1) {
        fact[0] = 1;
        for (int i = 1; i <= n; ++i) fact[i] = fact[i-1]
            * i % MOD;
        invFact[n] = modPow(fact[n], MOD - 2);
        for (int i = n; i; --i) invFact[i-1] =
            invFact[i] * i % MOD;
    }
    long long nCr(int n, int r) const {
        if (r < 0 || r > n) return 0;
        return fact[n] * invFact[r] % MOD * invFact[n-r]
            % MOD;
    }
    long long nPr(int n, int r) const { return r < 0 ||
        r > n ? 0 : fact[n] * invFact[n-r] % MOD; }
};

// Stars and bars: ways to distribute n identical items
    into k nonempty groups = C(n-1,k-1).
// Derangements: D[0]=1, D[1]=0,
    D[n]=(n-1)(D[n-1]+D[n-2]).
// Catalan: C[n]=C(2n,n)/(n+1), valid modulo prime when
    factorial range is supported.
// Stirling 2nd kind: S(n,k)=k*S(n-1,k)+S(n-1,k-1),
    S(0,0)=1.
// Code: vector<vector<long long>> S(n+1,vector<long
    long>(k+1)); S[0][0]=1;
// for i=1..n, j=1..min(i,k):
    S[i][j]=(j*S[i-1][j]+S[i-1][j-1]) % MOD.
// PIE: count union = sum(nonempty subsets A)
    (-1)^(|A|+1) count(intersection A).
// Probability: favorable/total; conditional P(A|B)=P(A
    and B)/P(B). Variance: Var(X)=E[X^2] - (E[X])^2.
// Expected value: E[X]=sum_i E[X_i] (linearity holds
    even if dependent). Coupon collector: n * H_n.
// DAG transitions: E[u] = cost + sum(p_v * E[v]);
    cyclic states require Gaussian elimination (see
    Expected Value DP).
// Indicator trick: write count as sum of 0/1
    indicators, then take expectations.
```

7.3 Euler Phi and Mobius [24 lines] - 4089BFF8

```
// Euler phi and Mobius values for every x <= n, linear
    sieve O(n).
vector<int> phiMobiusSieve(int n, vector<int>& mu) {
    vector<int> phi(n+1), lp(n+1), primes;
        mu.assign(n+1,0);
    if(n>=1) phi[1]=mu[1]=1;
    for(int i=2;i<=n;++i) {
        if(!lp[i]) { lp[i]=i; primes.push_back(i);
            phi[i]=i-1; mu[i]=-1; }
```

```
        for(int p:primes) {
            if(p>lp[i] || 1LL*i*p>n) break;
            lp[i*p]=p;
            if(i%p==0) { phi[i*p]=phi[i]*p; mu[i*p]=0;
                break; }
            phi[i*p]=phi[i]*(p-1); mu[i*p]=-mu[i];
        }
    }
    return phi;
}
long long eulerPhi(long long n) {
    long long result=n;
    for(long long p=2;p*p<=n;++p) if(n%p==0) {
        while(n%p==0) n/=p;
        result-=result/p;
    }
    if(n>1) result-=result/n;
    return result;
}
```

7.4 Euler Theorem and Power Towers [41 lines] - 90E18453

```
// Power Tower modulo m using generalized Euler's
    reduction theorem:
// a^b mod m = a^(b < phi(m) ? b : b % phi(m) + phi(m))
    mod m. Works for ANY a, m, even non-coprime.
#include <bits/stdc++.h>
using namespace std;

long long phi(long long n) {
    long long res = n;
    for (long long i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            res -= res / i;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}

// Computes (base ^ exp) with mod flag: if >= mod,
    returns (val % mod) + mod
long long powerModWithFlag(long long base, long long
    exp, long long mod) {
    long long res = 1;
    bool over = false;
    if (base >= mod) { base %= mod; over = true; }
    while (exp > 0) {
        if (exp & 1) {
            res *= base;
            if (res >= mod) { res %= mod; over = true; }
        }
        base *= base;
        if (base >= mod) { base %= mod; over = true; }
        exp >>= 1;
    }
    return over ? res + mod : res;
}

// Solves a[i] ^ a[i+1] ^ ... ^ a[n-1] mod m
long long powerTower(const vector<long long>& a, int i,
    long long m) {
    if (i == (int)a.size() || m == 1) return 1;
    long long p = phi(m);
    long long exp = powerTower(a, i + 1, p);
    return powerModWithFlag(a[i], exp, m) % m;
}
```

7.5 Exact GCD Counting [16 lines] - F3CAC16C

```
// Count pairs from two arrays with each exact gcd. O(V
    log V), V=max value.
// divisors[d] counts pairs where both values are
    divisible by d.
vector<long long> exactGcdPairs(const vector<int>&
    a,const vector<int>& b,int maxValue) {
    vector<long long> ca(maxValue+1), cb(maxValue+1),
        exact(maxValue+1);
    for (int x:a) ++ca[x];
    for (int x:b) ++cb[x];
    for (int d=maxValue;d>=1;--d) {
        long long count=0;
        for (int x=d;x<=maxValue;x+=d) count+=ca[x];
        long long other=0;
        for (int x=d;x<=maxValue;x+=d) other+=cb[x];
        exact[d]=count*other;
        for (int x=2*d;x<=maxValue;x+=d)
            exact[d]-=exact[x];
    }
    return exact; // exact[d] counts ordered cross-array
        pairs with gcd exactly d.
}
```

7.6 Floor Sum and BSGS [63 lines] - 1C09CEE5

```
// Floor Sum, Primitive Root, and Baby-Step Giant-Step
    (BSGS)
#include <bits/stdc++.h>
using namespace std;

// 1. Floor Sum: sum_{i=0}^{n-1} floor((a * i + b) / m)
    in O(log(n + m)). AtCoder Library.
long long floorSum(long long n, long long m, long long
    a, long long b) {
    long long ans = 0;
    if (a >= m) { ans += (n - 1) * n / 2 * (a / m); a
        %= m; }
    if (b >= m) { ans += n * (b / m); b %= m; }
    long long yMax = (a * n + b) / m, xMax = yMax * m -
        b;
    if (yMax == 0) return ans;
    ans += (n - (xMax + a - 1) / a) * yMax;
    ans += floorSum(yMax, a, m, (a - xMax % a) % a);
    return ans;
}

// 2. Primitive Root: smallest generator modulo prime p
long long modPow(long long a, long long b, long long
    mod) {
    long long res = 1; a %= mod;
    for (; b; b >>= 1, a = a * a % mod) if (b & 1) res
        = res * a % mod;
    return res;
}
int primitiveRoot(int p) {
    vector<int> fact;
    int phi = p - 1, n = phi;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0) n /= i;
        }
    }
    if (n > 1) fact.push_back(n);
    for (int res = 2; res <= p; ++res) {
        bool ok = true;
        for (int factor : fact) {
            if (modPow(res, phi / factor, p) == 1) { ok
                = false; break; }
        }
        if (ok) return res;
    }
    return -1;
}

// 3. Baby-Step Giant-Step (BSGS): solve a^x = b (mod m)
    for coprime a, m in O(sqrt(m))
long long bsgs(long long a, long long b, long long m) {
    a %= m; b %= m;
    long long n = sqrt(m) + 1;
    unordered_map<long long, long long> vals;
    long long cur = 1;
    for (long long p = 0; p <= n; ++p) {
        vals[cur * b % m] = p;
        cur = cur * a % m;
    }
    long long an = cur; // a^n
    cur = 1;
    for (long long q = 1; q <= n; ++q) {
        cur = cur * an % m;
        if (vals.count(cur)) {
            long long ans = q * n - vals[cur];
            return ans;
        }
    }
    return -1; // no solution
}
```

7.7 Gaussian Elimination [71 lines] - E05CFA68

```
// Gaussian Elimination: real (double) and modular (mod
    prime p) linear equation solvers.
#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-9;
const int INF = 2; // 0: no solution, 1: unique
    solution, INF: infinitely many

// 1. Double precision solver: a is n x (m + 1)
    augmented matrix; ans contains solution
int gaussDouble(vector<vector<double>> a,
    vector<double>& ans) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n;
        ++col) {
        int sel = row;
        for (int i = row; i < n; ++i)
            if (abs(a[i][col]) > abs(a[sel][col])) sel
                = i;
        if (abs(a[sel][col]) < EPS) continue;
        swap(a[sel], a[row]);
        where[col] = row;
        for (int i = 0; i < n; ++i) if (i != row) {
            double c = a[i][col] / a[row][col];
            for (int j = col; j <= m; ++j) a[i][j] -=
                a[row][j] * c;
        }
        ++row;
```

```
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; ++i)
        if (where[i] != -1) ans[i] = a[where[i]][m] /
            a[where[i]][i];
    for (int i = 0; i < n; ++i) {
        double sum = 0;
        for (int j = 0; j < m; ++j) sum += ans[j] *
            a[i][j];
        if (abs(sum - a[i][m]) > EPS) return 0;
    }
    for (int i = 0; i < m; ++i) if (where[i] == -1)
        return INF;
    return 1;
}

// 2. Modulo Prime p solver: powerMod and modInverse
    helper
long long modPow(long long a, long long b, long long
    mod) {
    long long res = 1; a %= mod;
    for (; b; b >>= 1, a = a * a % mod) if (b & 1) res
        = res * a % mod;
    return res;
}

int gaussMod(vector<vector<int>> a, vector<int>& ans,
    int mod) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n;
        ++col) {
        int sel = row;
        for (int i = row; i < n; ++i) if (a[i][col] > 0)
            { sel = i; break; }
        if (a[sel][col] == 0) continue;
        swap(a[sel], a[row]);
        where[col] = row;
        int inv = modPow(a[row][col], mod - 2, mod);
        for (int j = col; j <= m; ++j) a[row][j] = 1LL
            * a[row][j] * inv % mod;
        for (int i = 0; i < n; ++i) if (i != row &&
            a[i][col]) {
            int c = a[i][col];
            for (int j = col; j <= m; ++j)
                a[i][j] = (a[i][j] - 1LL * a[row][j] * c
                    % mod + mod) % mod;
        }
        ++row;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; ++i) if (where[i] != -1)
        ans[i] = a[where[i]][m];
    for (int i = 0; i < n; ++i) {
        int sum = 0;
        for (int j = 0; j < m; ++j) sum = (sum + 1LL *
            ans[j] * a[i][j]) % mod;
        if (sum != a[i][m]) return 0;
    }
    for (int i = 0; i < m; ++i) if (where[i] == -1)
        return INF;
    return 1;
}
```

7.8 Linear Diophantine Equation [15 lines] - 8747D6A7

```
// Solve a*x+b*y=c. Returns false if no integer solution
    exists.
long long egcd(long long a,long long b,long long& x,long
    long& y) {
    if(!b) { x=1; y=0; return a; }
    long long x1,y1,g=egcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g;
}
bool solveDiophantine(long long a,long long b,long long
    c,long long& x,long long& y) {
    if(a==0&&b==0){x=y=0;return c==0;}
    long long g=egcd(llabs(a),llabs(b),x,y);
    if(c%g) return false;
    x*=c/g; y*=c/g;
    if(a<0) x=-x; if(b<0) y=-y;
    return true;
}
// All solutions: x += (b/g)*t, y -= (a/g)*t.
```

7.9 Lucas Theorem [21 lines] - 937ED62C

```
// nCr modulo a prime p, even when n >= p. TIME: O(p +
    log_p n).
// Requires p small enough to precompute factorials up
    to p-1.
long long modPow(long long a,long long e,long long mod)
    {
    long long r=1;
    for (;e;e>>=1,a=a*a%mod) if (e&1) r=r*a%mod;
    return r;
}
long long chooseSmall(long long n,long long r,const
    vector<long long>& fact,long long p) {
    if (r<0 || r>n) return 0;
    return fact[n]*modPow(fact[r]*fact[n-r]%p,p-2,p)%p;
}
long long lucas(long long n,long long r,long long
    p,const vector<long long>& fact) {
    long long ans=1;
    while (n || r) {
        long long ni=n%p, ri=r%p;
        if (ri>ni) return 0;
        ans=ans*chooseSmall(ni,ri,fact,p)%p;
        n/=p; r/=p;
    }
    return ans;
}
```

7.10 Miller-Rabin Pollard-Rho [43 lines] - 0FC59EF1

```
// Deterministic primality test and factorization for
    64-bit unsigned integers.
using u64 = unsigned long long;
using u128 = __uint128_t;
u64 mulMod(u64 a,u64 b,u64 mod) { return (u128)a*b%mod;
    }
u64 powMod64(u64 a,u64 e,u64 mod) {
    u64 r=1;
    for (;e;e>>=1,a=mulMod(a,a,mod)) if (e&1)
        r=mulMod(r,a,mod);
    return r;
}
bool isPrime64(u64 n) {
    if (n<2) return false;
    for (u64 p:{2ULL,3ULL,5ULL,7ULL,11ULL,13ULL,17ULL,
        19ULL,23ULL,29ULL,31ULL,37ULL}) {
        if (n%p==0) return n==p;
    }
    u64 d=n-1, s=0; while (!(d&1)) d>>=1,++s;
    for (u64 a:{2ULL,325ULL,9375ULL,28178ULL,450775ULL,
        9780504ULL,1795265022ULL}) {
        if (a%n==0) continue;
        u64 x=powMod64(a%n,d,n);
        if (x==1 || x==n-1) continue;
        bool composite=true;
        for (int r=1;r<s;++r) { x=mulMod(x,x,n); if
            (x==n-1) { composite=false; break; } }
        if (composite) return false;
    }
    return true;
}
auto seedClock = chrono::steady_clock::now();
mt19937_64 rng64(seedClock.time_since_epoch().count());
u64 pollardRho(u64 n) {
    if (n%2==0) return 2;
    if (n%3==0) return 3;
    while (true) {
        u64 c=rng64()%(n-1)+1, x=rng64()%(n-2)+2, y=x,
            d=1;
        auto f=[&](u64 v){ return
            (u64)(((u128)mulMod(v,v,n)+c)%n); };
        while (d==1) { x=f(x); y=f(f(y));
            d=gcd(x>y?x-y:y-x,n); }
        if (d!=n) return d;
    }
}
void factor64(u64 n,map<u64,int>& factors) {
    if (n==1) return;
    if (isPrime64(n)) { ++factors[n]; return; }
    u64 d=pollardRho(n);
    factor64(d,factors); factor64(n/d,factors);
}
```

7.11 Modular Inverse [13 lines] - 6D6C77FA

```
// Extended Euclid works for any modulus when
    gcd(a,m)=1.
long long extendedGcd(long long a,long long b,long long&
    x,long long& y) {
    if(!b) { x=1; y=0; return a; }
    long long x1,y1,g=extendedGcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1; return g;
}
long long modInverse(long long a,long long mod) {
    long long x,y,g=extendedGcd(a,mod,x,y);
    if(g!=1 && g!=-1) return -1; // no inverse
    x/=g;
    return (x%mod+mod)%mod;
}
// For prime MOD and a%MOD != 0: inverse = a^(MOD-2) mod
    MOD.
```

7.12 NTT [53 lines] - 22422027

```
// Number Theoretic Transform (NTT) modulo 998244353.
    O(n log n) polynomial multiplication.
#include <bits/stdc++.h>
using namespace std;

const int MOD = 998244353;
const int G = 3; // primitive root for 998244353

long long power(long long base, long long exp) {
    long long res = 1; base %= MOD;
    for (; exp > 0; exp >>= 1, base = base * base % MOD)
        if (exp & 1) res = res * base % MOD;
```

```
    return res;
}
long long modInverse(long long n) { return power(n, MOD
    - 2); }

void ntt(vector<int>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long wlen = power(G, (MOD - 1) / len);
        if (invert) wlen = modInverse(wlen);
        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < len / 2; ++j) {
                long long u = a[i + j];
                long long v = a[i + j + len / 2] * w %
                    MOD;
                a[i + j] = (u + v) % MOD;
                a[i + j + len / 2] = (u - v + MOD) %
                    MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (invert) {
        long long n_inv = modInverse(n);
        for (int& x : a) x = x * n_inv % MOD;
    }
}

vector<int> multiply(vector<int> a, vector<int> b) {
    int n = 1;
    int target = a.size() + b.size() - 1;
    while (n < target) n <<= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; ++i) a[i] = 1LL * a[i] * b[i]
        % MOD;
    ntt(a, true);
    a.resize(target);
    return a;
}
```

7.13 Sieve [42 lines] - B1AB1E33

```
// Sieve of Eratosthenes, Linear Sieve / SPF, and
    Segmented Sieve
#include <bits/stdc++.h>
using namespace std;

// Sieve with Smallest Prime Factor (SPF) for O(log n)
    prime factorization
vector<int> primes, spf;
void sieveSPF(int n) {
    spf.assign(n + 1, 0);
    for (int i = 2; i <= n; ++i) {
        if (!spf[i]) { spf[i] = i; primes.push_back(i);
            }
        for (int p : primes) {
            if (p > spf[i] || 1LL * i * p > n) break;
            spf[i * p] = p;
        }
    }
}
vector<int> getFactors(int x) {
    vector<int> f;
    while (x > 1) { f.push_back(spf[x]); x /= spf[x]; }
    return f;
}

// Segmented Sieve: primes in [L, R], where R - L <=
    1e7, R <= 1e12
vector<long long> segmentedSieve(long long L, long long
    R) {
    long long lim = sqrt(R);
    vector<char> mark(lim + 1, 1);
    vector<long long> basePrimes;
    for (long long i = 2; i <= lim; ++i) {
        if (mark[i]) {
            basePrimes.push_back(i);
            for (long long j = i * i; j <= lim; j += i)
                mark[j] = 0;
        }
    }
    vector<char> isP(R - L + 1, 1);
    if (L == 1) isP[0] = 0;
    for (long long p : basePrimes) {
        for (long long j = max(p * p, (L + p - 1) / p *
            p); j <= R; j += p)
            isP[j - L] = 0;
    }
    vector<long long> res;
    for (long long i = 0; i <= R - L; ++i) if (isP[i])
        res.push_back(L + i);
    return res;
}
```

7.14 Stirling Numbers [7 lines] - D3007645

```
// Stirling numbers of the second kind S(n,k) modulo
    MOD.
vector<vector<long long>> stirlingSecondKind(int n,int
    k,long long MOD) {
    vector<vector<long long>> s(n+1,vector<long
        long>(k+1)); s[0][0]=1;
    for(int i=1;i<=n;++i) for(int j=1;j<=min(i,k);++j)
        s[i][j]=(j*s[i-1][j]+s[i-1][j-1])%MOD;
    return s;
}
```

7.15 Sum of Divisors [12 lines] - FCF9A6F7

```
// Sum of positive divisors of n from prime
    factorization: n=product p^e.
long long sumDivisors(long long n) {
    long long answer=1;
    for(long long p=2;p<=n/p;++p) if(n%p==0) {
        long long term=1, power=1;
        while(n%p==0) { n/=p; power*=p; term+=power; }
        answer*=term;
    }
    if(n>1) answer*=1+n;
    return answer;
}
// May overflow long long; use __int128 or modular
    arithmetic for large answers.
```

8 Misc

8.1 Bit hacks [12 lines] - 1398A0E8

```
# x & -x is the least bit in x.
# iterate over all the subsets of the mask
for (int s=m; ; s=(s-1)&m) {
 ... you can use s ...
 if (s==0)  break;
}
# c = x&-x, r = x+c; (((r^x) >> 2)/c) | r is the
next number after x with the same number of bits set.
# __builtin_popcount(x) //number of ones in binary
  __builtin_popcountll(x) // for long long
# __builtin_clz(x) // number of leading zeros
  __builtin_ctz(x) // number of trailing zeros, they
      also have long long version
```

8.2 Contest Checklist and Complexity Table [33 lines] - 064C8435

```
1. INPUT SIZE (N) TO TIME COMPLEXITY GUIDE (for ~1
    second limit / ~10^8 operations):
- N <= 10        : O(N!) or O(N^6) - Permutations, brute
    force, backtracking
- N <= 20..22    : O(2^N * N) - Bitmask DP,
    meet-in-the-middle (2^(N/2)), SOS DP
- N <= 100       : O(N^4) - Floyd-Warshall variants,
    4-nested loops
- N <= 400..500  : O(N^3) - Floyd-Warshall, matrix
    exponentiation (N^3 log K), standard MCMF
- N <= 2000..3000: O(N^2) - 2D DP, all pairs, basic
    geometry, Dinic on dense graphs
- N <= 10^5      : O(N sqrt(N)) - Mo's algorithm, sqrt
    decomposition, block queries
- N <= 10^6      : O(N log N) or O(N log^2 N) - Sorting,
    Segment Tree, DSU, HLD, Suffix Array, NTT
- N <= 10^7..10^8: O(N) - Linear sieve, two pointers,
    prefix sums, KMP, BFS/DFS, basic array scan
- N > 10^9       : O(log N) or O(1) - Binary search,
    matrix power, math formulas, GCD, Euler's totient

2. BEFORE YOU SUBMIT CHECKLIST:
[ ] Array Bounds & Sizes:
    - 0-indexed vs 1-indexed consistency. Check row/col
        and n/m swaps.
    - SegTree: 4*N; Treap/Trie/SAM: 2*N nodes; Mo on
        Tree: 2*N doubled tour; 2-SAT: 2*N.
[ ] Integer Overflow:
    - Are sums, products, distances, capacities declared
        as long long?
    - Cast intermediate products before multiplication:
        1LL * a * b % MOD.
    - Use __int128 for intermediate values exceeding
        64-bit bounds.
[ ] Modular Arithmetic:
    - Subtraction modulo M: always do (a - b % M + M) %
        M.
    - Avoid division by zero: divide means multiplying
        by modInverse(b, MOD).
[ ] Multi-test Reset:
    - Clean up EVERY global variable, vector, tree, and
        graph: g.clear(), seen.assign(), etc.
    - Do NOT call memset on huge buffers inside the test
        case loop; clear only up to N.
[ ] Extreme Corner Cases:
    - N = 0, N = 1, N = 2.
    - Empty arrays/strings, negative weights, zero
        coordinates, identical values.
    - Disconnected graph, trees with a single edge,
        self-loops, multiple parallel edges.
```

```
[ ] Interactive Problems:
    - Always flush output: cout << endl (or
      fflush(stdout)) after printing answer/query.
[ ] Floating Point:
    - Use double / long double; compare with EPS (1e-9);
      avoid == on floats.
```

8.3 STL Reference [28 lines] - 14293C11

```
// Handy STL patterns. Include <bits/stdc++.h> and use
    namespace std.
vector<int> a{1,2,3,4}, b{2,4,6}, v{1,3,5};
set<int> s{1,4,7};
// nth_element: puts the element that would be at k in
    sorted order at position k.
// Average O(n); elements on either side are not fully
    sorted.
// nth_element(a.begin(), a.begin()+k, a.end()); // k-th
    smallest, 0-indexed

// Iterator helpers: distance(first,last),
    advance(it,k), next(it), prev(it).
// For vector iterators these are O(1); for set/map
    iterators they are O(n).
auto it = lower_bound(v.begin(), v.end(), x); // sorted
    vector: first value >= x
auto jt = next(s.begin());                    // ordered
    containers: move by one

// Set algorithms require sorted ranges and write into
    an output iterator.
vector<int> out;
set_intersection(a.begin(),a.end(),b.begin(),b.end(),
    back_inserter(out));
// Also: set_union, set_difference,
    set_symmetric_difference.

// bitset: fixed compile-time number of bits, useful for
    compact state sets.
bitset<1000> mask; mask.set(3); mask.flip(5); bool has
    = mask.test(3);

// tuple: group values of different types. Structured
    binding since C++17.
tuple<int,string,long long> item={1,"name",42};
auto [id,name,value]=item;

// transform: apply a function elementwise. partition:
    split by a predicate.
vector<int> squares(a.size());
transform(a.begin(),a.end(),squares.begin(),[](int
    x){return x*x;});
auto middle=partition(a.begin(),a.end(),[](int x){return
    x%2==0;});
```

8.4 Template [67 lines] - CA5EB085

```
// Contest Template: compile flags, fast IO, debug
    macro, pb_ds ordered_set, custom hash, __int128,
    stress-test
// Compile: g++ -O2 -std=c++17 -Wall -Wextra -Wshadow
    -fsanitize=address,undefined -DLOCAL sol.cpp -o sol
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
using ll = long long;
using pii = pair<int, int>;

// 1. Policy Based Data Structure: Ordered Set
template <typename T>
using ordered_set = tree<T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
// .find_by_order(k): iterator to k-th smallest
    (0-indexed); .order_of_key(x): count strictly < x

// 2. Custom anti-hack hash for unordered_map /
    unordered_set
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();

        return splitmix64(x + FIXED_RANDOM);
    }
};
template <typename K, typename V> using safe_map =
    unordered_map<K, V, custom_hash>;

// 3. __int128 IO
void print128(__int128 x) {
    if (x < 0) { cout << '-'; x = -x; }
    if (x > 9) print128(x / 10);
    cout << (char)('0' + (x % 10));
}
__int128 read128() {
    __int128 x = 0; int f = 1; char ch = cin.get();
    while (!isdigit(ch)) { if (ch == '-') f = -1; ch =
        cin.get(); }
    while (isdigit(ch)) { x = x * 10 + ch - '0'; ch =
        cin.get(); }
    return x * f;
}

// 4. Debug macro
#ifdef LOCAL
#define dbg(...) cerr << "[" << #__VA_ARGS__ << "]:",
    debug_out(__VA_ARGS__)
void debug_out() { cerr << endl; }
template <typename Head, typename... Tail>
void debug_out(Head H, Tail... T) { cerr << " " << H;
    debug_out(T...); }
#else
#define dbg(...)
#endif

// 5. Stress test bash script:
  // for((i=1;;i++)); do ./gen
    i > in; ./sol < in > out1; ./brute < in >
    out2; diff - wout1out2||break; echo"Testi OK"; done

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    // cin >> T;
    while (T--) {
        // solve();
    }
    return 0;
}
```

8.5 XOR Hashing [15 lines] - 5F1277A0

```
// WHEN: randomized fingerprints for set-like data.
    TIME: O(1) per key. ASSUMES: collisions remain
    possible; not cryptographic.
// Randomized XOR fingerprints for sets/multisets of
    small integer keys.
// Collision probability is small, never zero; do not
    use as a cryptographic hash.
uint64_t splitmix64(uint64_t x) {
    x+=0x9e3779b97f4a7c15ULL;
        x=(x^(x>>30))*0xbf58476d1ce4e5b9ULL;
    x=(x^(x>>27))*0x94d049bb133111ebULL; return
        x^(x>>31);
}
struct XorHash {
    uint64_t seed = chrono::steady_clock::now()
        .time_since_epoch()
        .count();
    uint64_t operator()(uint64_t x) const { return
        splitmix64(x+seed); }
};
// Set fingerprint: XOR hash(x) for every element;
    add/remove toggles the same value.
// For multisets, XOR loses even multiplicities; use a
    sum of random hashes instead.
```

9 Searching

9.1 Basic Array Patterns [38 lines] - 1DE99FBA

```
// 1D prefix sums: inclusive range sum [l,r] in O(1)
    after O(n) build.
vector<long long> buildPrefix(const vector<int>& a) {
    vector<long long> pref(a.size()+1);
    for (int i=0;i<(int)a.size();++i)
        pref[i+1]=pref[i]+a[i];
    return pref;
}
long long rangeSum(const vector<long long>& pref,int
    l,int r) {
    return pref[r+1]-pref[l];
}

// Difference array: add value to every index in
    inclusive [l,r].
// After all updates, prefix-sum diff once to restore
    the final values.
void rangeAdd(vector<long long>& diff,int l,int r,long
    long value) {
    diff[l]+=value;
    if (r+1<(int)diff.size()) diff[r+1]-=value;
}

// Sliding window for nonnegative values: longest
    subarray with sum <= limit.
int longestWindowAtMost(const vector<int>& a,long long
    limit) {
    int left=0,best=0; long long sum=0;
    for (int right=0;right<(int)a.size();++right) {
        sum+=a[right];
```

```
        while (left<=right && sum>limit) sum-=a[left++];
        best=max(best,right-left+1);
    }
    return best;
}

// Two pointers on a sorted array: determine whether two
    values sum to target.
bool hasPairSum(const vector<long long>& a,long long
    target) {
    int l=0,r=(int)a.size()-1;
    while (l<r) {
        long long sum=a[l]+a[r];
        if (sum==target) return true;
        if (sum<target) ++l; else --r;
    }
    return false;
}
```

9.2 Basic Binary Search [27 lines] - 2333BA5B

```
// lower_bound on a sorted vector: first index i with
    a[i] >= x, or n.
int firstAtLeast(const vector<long long>& a,long long x)
    {
    int l=0,r=a.size();
    while (l<r) { int mid=l+(r-l)/2; if (a[mid]<x)
        l=mid+1; else r=mid; }
    return l;
}

// Last true for a monotone predicate: P(l)=true,
    P(r)=false; returns l.
template<class Predicate>
long long lastTrue(long long l,long long r,Predicate P)
    {
    while (r-l>1) {
        long long mid=l+(r-l)/2;
        if (P(mid)) l=mid; else r=mid;
    }
    return l;
}

// Binary search on answer: find the minimum feasible
    value.
// Establish infeasible low and feasible high bounds
    before calling the loop.
template<class Predicate>
long long firstTrue(long long low,long long
    high,Predicate feasible) {
    while (high-low>1) {
        long long mid=low+(high-low)/2;
        if (feasible(mid)) high=mid; else low=mid;
    }
    return high;
}
```

9.3 Range and Search Techniques [41 lines] - 8926C3DB

```
// 2D prefix sum: build then sum inclusive rectangle
    (x1,y1)..(x2,y2).
vector<vector<long long>> prefix2D(const
    vector<vector<int>>& a) {
    int n = a.size(), m = n ? a[0].size() : 0;
    vector<vector<long long>> p(n + 1, vector<long
        long>(m + 1));
    for (int i = 1; i <= n; ++i) for (int j = 1; j <=
        m; ++j)
        p[i][j] = a[i-1][j-1] + p[i-1][j] + p[i][j-1] -
            p[i-1][j-1];
    return p;
}
long long rectangleSum(const vector<vector<long long>>&
    p, int x1, int y1, int x2, int y2) {
    return p[x2+1][y2+1] - p[x1][y2+1] - p[x2+1][y1] +
        p[x1][y1];
}

// Ternary search for a unimodal function on a closed
    interval.
double ternarySearch(double l, double r,
    function<double(double)> f) {
    for (int it = 0; it < 100; ++it) {
        double m1 = l + (r-l)/3, m2 = r - (r-l)/3;
        if (f(m1) < f(m2)) l = m1; else r = m2; //
            maximum; reverse for minimum
    }
    return (l+r)/2;
}

// Meet in the middle: all subset sums, O(2^(n/2)) time
    and memory per half.
vector<long long> subsetSums(const vector<long long>&
    a, int l, int r) {
    vector<long long> sums{0};
    for (int i = l; i < r; ++i) {
        int old = sums.size();
        for (int j = 0; j < old; ++j)
            sums.push_back(sums[j] + a[i]);
    }
    return sums;
}
// 4-sum existence with four distinct indices, expected
    O(n^2) time and memory.
bool fourSum(const vector<long long>& a, long long
    target) {
    unordered_set<long long> earlierPairSums;
    int n=a.size();
    for (int i=0;i<n;++i) {
        for (int j=i+1;j<n;++j)
            if (earlierPairSums.count(target-a[i]-a[j]))
                return true;
        for (int x=0;x<i;++x)
            earlierPairSums.insert(a[x]+a[i]);
    }
    return false;
}
```

10 String

10.1 Aho-Corasick [27 lines] - A686271F

```
// Lowercase a-z patterns; reports number of matches per
    pattern after scanning text.
struct AhoCorasick {
    struct Node { int next[26]; int link = 0;
        vector<int> out; Node() {
        fill(next,next+26,-1); } };
    vector<Node> t{Node()};
    void add(const string& s, int id) {
        int v=0; for(char c:s) { int x=c-'a';
            if(t[v].next[x]==-1) {
            t[v].next[x]=t.size(); t.emplace_back(); }
            v=t[v].next[x]; }
        t[v].out.push_back(id);
    }
    void build() {
        queue<int> q;
        for(int c=0;c<26;++c) { int u=t[0].next[c];
            if(u==-1) t[0].next[c]=0; else q.push(u); }
        while(!q.empty()) {
            int v=q.front(); q.pop();
            for(int id:t[t[v].link].out)
                t[v].out.push_back(id);
            for(int c=0;c<26;++c) {
                int u=t[v].next[c];
                if(u==-1)
                    t[v].next[c]=t[t[v].link].next[c];
                else { t[u].link=t[t[v].link].next[c];
                    q.push(u); }
            }
        }
    }
    vector<int> countMatches(const string& text,int
        patternCount) const {
        vector<int> count(patternCount); int v=0;
        for(char c:text) { v=t[v].next[c-'a']; for(int
            id:t[v].out) ++count[id]; }
        return count;
    }
};
```

10.2 KMP [17 lines] - A1049B24

```
vector<int> prefixFunction(const string& s) {
    vector<int> pi(s.size());
    for(int i=1;i<(int)s.size();++i) {
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) ++j;
        pi[i]=j;
    }
    return pi;
}
vector<int> findMatches(const string& text,const string&
    pattern) {
    if(pattern.empty()) return {};
    string s=pattern+'#'+text; vector<int>
        pi=prefixFunction(s), matches;
    for(int i=(int)pattern.size()+1;i<(int)s.size();++i)
        if(pi[i]==(int)pattern.size())
            matches.push_back(i-2*(int)pattern.size());
    return matches;
}
```

10.3 Lyndon and Hash61 [65 lines] - EC534088

```
// Lyndon Factorization, Minimal String Rotation, and
    Mersenne 61-bit Hash (2^61 - 1)
#include <bits/stdc++.h>
using namespace std;

// 1. Duval's Algorithm: O(n) Lyndon factorization of
    string s into s = w_1 w_2 ... w_k where w_1 >= ...
    >= w_k
vector<string> duval(const string& s) {
    int n = s.size(), i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
```

```
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
    return factorization;
}

// 2. Lexicographically Minimal String Rotation in O(n)
    (Booth's Algorithm)
int minRotation(string s) {
    s += s;
    int n = s.size(), i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) i += j - k;
    }
    return ans; // 0-based starting index in original
        string
}

// 3. Fast 61-bit Rolling Hash modulo Mersenne prime
    2^61 - 1 (unhackable, O(1) query)
struct Hash61 {
    static const uint64_t MOD = (1ULL << 61) - 1;
    static uint64_t mulMod(uint64_t a, uint64_t b) {
        __uint128_t prod = (__uint128_t)a * b;
        uint64_t res = (prod >> 61) + (prod & MOD);
        if (res >= MOD) res -= MOD;
        return res;
    }
    uint64_t BASE;
    vector<uint64_t> h, p;
    Hash61(const string& s) : h(s.size() + 1, 0),
        p(s.size() + 1, 1) {
        mt19937_64
            rng(chrono::steady_clock::now().time_since_epoch().count());

        BASE = uniform_int_distribution<uint64_t>(300,
            MOD - 2)(rng) | 1;
        for (int i = 0; i < (int)s.size(); ++i) {
            p[i + 1] = mulMod(p[i], BASE);
            h[i + 1] = mulMod(h[i], BASE) + (unsigned
                char)s[i] + 1;
            if (h[i + 1] >= MOD) h[i + 1] -= MOD;
        }
    }
    uint64_t get(int l, int r) const { // [l, r)
        uint64_t res = h[r] + MOD - mulMod(h[l], p[r -
            l]);
        return res >= MOD ? res - MOD : res;
    }
};
```

10.4 Manacher [15 lines] - FBB15D9E

```
// Longest palindromic substring length in O(n).
int longestPalindrome(const string& s) {
    string t="^";
    for(char c:s) t += string("#") + c;
    t += "#$";
    vector<int> p(t.size()); int
        center=0,right=0,best=0;
    for(int i=1;i+1<(int)t.size();++i) {
        int mirror=2*center-i;
        if(i<right) p[i]=min(right-i,p[mirror]);
        while(t[i+1+p[i]]==t[i-1-p[i]]) ++p[i];
        if(i+p[i]>right) center=i,right=i+p[i];
        best=max(best,p[i]);
    }
    return best;
}
```

10.5 Palindromic Tree [48 lines] - 6F246768

```
// Palindromic Tree (Eertree): O(n) distinct
    palindromes, lengths, occurrences, and transitions.
#include <bits/stdc++.h>
using namespace std;

struct PalindromicTree {
    struct Node {
        int len, link;
        int next[26];
        int count = 0; // occurrence count after prop
        Node(int len, int link) : len(len), link(link) {
            memset(next, 0, sizeof(next));
        }
    };
    vector<Node> tree;
    string s;
    int last, sz;

    PalindromicTree(int maxLen = 1) {
        tree.emplace_back(0, 1);    // 0: even root (len
            0)
        tree.emplace_back(-1, 1);   // 1: odd root (len
            -1)
        sz = 2; last = 0;
    }

    int getLink(int v, int pos) {
        while (pos - tree[v].len - 1 < 0 || s[pos -
            tree[v].len - 1] != s[pos])
            v = tree[v].link;
        return v;
    }

    void extend(int pos) {
        int cur = getLink(last, pos);
        int ch = s[pos] - 'a';
        if (!tree[cur].next[ch]) {
            int link = tree[getLink(tree[cur].link,
                pos)].next[ch];
            tree.emplace_back(tree[cur].len + 2, link ?
                link : 0);
            tree[cur].next[ch] = sz++;
        }
        last = tree[cur].next[ch];
        tree[last].count++;
    }

    void build(const string& str) {
        s = str;
        for (int i = 0; i < (int)s.size(); ++i)
            extend(i);
        // Propagate counts from longest to shortest
        for (int i = sz - 1; i >= 2; --i)
            tree[tree[i].link].count += tree[i].count;
    }
};
```

10.6 Rolling Hash [30 lines] - B055754E

```
// Substring rolling hash with randomized base to
    prevent anti-hash test generation.
// O(n) build, O(1) query on [l, r).
static mt19937_64
    hash_rng(chrono::steady_clock::now().time_since_epoch().

static const long long MOD1 = 1000000007, MOD2 =
    1000000009;
static const long long BASE1 =
    uniform_int_distribution<long long>(300, MOD1 -
    2)(hash_rng) | 1;
static const long long BASE2 =
    uniform_int_distribution<long long>(300, MOD2 -
    2)(hash_rng) | 1;

struct RollingHash {
    vector<long long> h, power;
    long long mod, base;
    RollingHash(const string& s, long long mod = MOD1,
        long long base = BASE1)
        : h(s.size() + 1), power(s.size() + 1, 1),
            mod(mod), base(base) {
        for (int i = 0; i < (int)s.size(); ++i) {
            power[i + 1] = power[i] * base % mod;
            h[i + 1] = (h[i] * base + (unsigned
                char)s[i] + 1) % mod;
        }
    }
    long long get(int l, int r) const { // [l, r)
        long long x = (h[r] - h[l] * power[r - l]) %
            mod;
        return (x + mod) % mod;
    }
};

struct DoubleRollingHash {
    RollingHash rh1, rh2;
    DoubleRollingHash(const string& s) : rh1(s, MOD1,
        BASE1), rh2(s, MOD2, BASE2) {}
    pair<long long, long long> get(int l, int r) const {
        return {rh1.get(l, r), rh2.get(l, r)};
    }
};
```

10.7 Suffix Array [29 lines] - 7C2108AD

```
// Suffix array + Kasai LCP; O(n log^2 n) build, O(n)
    LCP.
struct SuffixArray {
    vector<int> sa, rank, lcp;
    explicit SuffixArray(const string& s) {
        int n=s.size(); sa.resize(n); rank.resize(n);
            iota(sa.begin(),sa.end(),0);
        for(int i=0;i<n;++i) rank[i]=(unsigned
            char)s[i];
        vector<int> nextRank(n);
        for(int k=1;k<n;k*=2) {
            sort(sa.begin(),sa.end(),[&](int a,int b) {
```

```
            if(rank[a]!=rank[b]) return
                rank[a]<rank[b];
            return (a+k<n?rank[a+k]:-1)<(b+k
                <n?rank[b+k]:-1);
        });
        if(n) nextRank[sa[0]]=0;
        for(int i=1;i<n;++i) {
            int a=sa[i-1],b=sa[i];
            nextRank[b]=nextRank[a]+(rank[a]!=
                rank[b] ||
                (a+k<n?rank[a+k]:-1)!=(b+k
                <n?rank[b+k]:-1));
        }
        rank.swap(nextRank);
        if(n && rank[sa[n-1]]==n-1) break;
    }
    vector<int> pos(n); for(int i=0;i<n;++i)
        pos[sa[i]]=i;
    lcp.assign(max(0,n-1),0); int h=0;
    for(int i=0;i<n;++i) {
        int r=pos[i]; if(r==n-1) { h=0; continue; }
        int j=sa[r+1]; while(i+h<n && j+h<n &&
            s[i+h]==s[j+h]) ++h;
        lcp[r]=h; if(h) --h;
    }
  }
};
```

10.8 Suffix Automaton [58 lines] - 9C63B26F

```
// Suffix Automaton (SAM): O(n) construction and linear
    size DAG of all substrings.
#include <bits/stdc++.h>
using namespace std;

struct SuffixAutomaton {
    struct State {
        int len, link;
        map<char, int> next;
    };
    vector<State> st;
    int sz, last;

    SuffixAutomaton(int maxLen = 1) { init(maxLen); }
    void init(int maxLen = 1) {
        st.clear();
        st.resize(maxLen * 2);
        st[0].len = 0;
        st[0].link = -1;
        sz = 1;
        last = 0;
    }

    void extend(char c) {
        int cur = sz++;
        st[cur].len = st[last].len + 1;
        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = sz++;
                st[clone].len = st[p].len + 1;
                st[clone].next = st[q].next;
                st[clone].link = st[q].link;
                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }

    // Number of distinct substrings in O(states)
    long long countDistinctSubstrings() const {
        long long total = 0;
        for (int i = 1; i < sz; ++i) total += st[i].len
            - st[st[i].link].len;
        return total;
    }
};
```

10.9 Trie [15 lines] - 48982540

```
// Lowercase a-z trie.
struct Trie {
    struct Node { int next[26]; bool terminal=false;
        Node() { fill(next,next+26,-1); } };
    vector<Node> t{Node()};
    void insert(const string& s) {
        int v=0;
        for(char c:s) { int x=c-'a';
            if(t[v].next[x]==-1) {
            t[v].next[x]=t.size(); t.emplace_back(); }
            v=t[v].next[x]; }
        t[v].terminal=true;
    }
    bool contains(const string& s) const {
        int v=0;
        for(char c:s) { int x=c-'a';
            if(t[v].next[x]==-1) return false;
            v=t[v].next[x]; }
        return t[v].terminal;
    }
};
```

10.10 Z-Algorithm [10 lines] - 7FEC0BF7

```
vector<int> zFunction(const string& s) {
    int n=s.size(); vector<int> z(n); int l=0,r=0;
    for(int i=1;i<n;++i) {
        if(i<r) z[i]=min(r-i,z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) ++z[i];
        if(i+z[i]>r) l=i,r=i+z[i];
    }
    if(n) z[0]=n;
    return z;
}
```

11 Basic Math and Formula Sheet

11.1 Integer Arithmetic

- Euclidean algorithm: $\gcd(a,b) = \gcd(b, a \bmod b)$; $\operatorname{lcm}(a,b) = |ab|/\gcd(a,b)$ (divide first to reduce overflow).
- Divisibility: if $d \mid a$ and $d \mid b$, then $d \mid (xa + yb)$ for integers x, y.
- For $n > 0$, the number of divisors is $\prod(e_i+1)$ and their sum is $\prod(1 + p_i + \cdots + p_i^{e_i})$ when $n = \prod p_i^{e_i}$.

11.2 Sums and Counting

- $1+2+\cdots+n = n(n+1)/2$; $\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$; $\sum_{i=1}^{n} i^3 = [n(n+1)/2]^2$.
- Arithmetic progression: $S_n = n(a_1 + a_n)/2$. Geometric progression: $1 + r + \cdots + r^n = (r^{n+1} - 1)/(r - 1)$ for $r \neq 1$.
- $\binom{n}{k} = \binom{n}{n-k}$ and $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.
- Pigeonhole principle: placing more than k objects into k boxes forces some box to contain at least two objects.

11.3 Modular Arithmetic

- Normalize possibly negative x by $(x\%m + m)\%m$ for $m > 0$.
- $(a + b) \bmod m$, $(a - b) \bmod m$, and $(ab) \bmod m$ can be computed from residues; use a wide integer type for products.
- Division modulo m means multiplication by an inverse. a^{-1} exists exactly when $\gcd(a, m) = 1$.
- For prime p and $p \nmid a$, Fermat gives $a^{p-1} \equiv 1 \pmod{p}$ and $a^{-1} \equiv a^{p-2} \pmod{p}$.

11.4 Bit and Geometry Reminders

- $x\&(-x)$ isolates the lowest set bit; $x\&(x-1)$ removes it; a positive power of two satisfies $(x\&(x-1)) = 0$.
- Squared Euclidean distance is $(x_1 - x_2)^2 + (y_1 - y_2)^2$; compare it instead of taking square roots.
- For integer points, the cross product gives orientation; polygon area is half the absolute shoelace sum. See Basic Geometry.cpp.

11.5 Combinatorics

- $\sum_{k=0}^{n} \binom{n-k}{k} = Fib_{n+1}$
- $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- $k\binom{n}{k} = n\binom{n-1}{k-1}$
- Number of binary sequences of length n such that no two 0's are adjacent $= Fib_{n+1}$
- Number of non-negative solution of $x_1 + x_2 + x_3 + ... + x_k = n$ is $\binom{n+k-1}{n}$

11.5.1 Catalan Number

- $C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$
- $C_0 = 1, C_1 = 1, C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$
- $1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786$
- Number of correct bracket sequences consisting of n opening brackets.
- Number of ways to completely parenthesize n+1 factors.
- The number of triangulations of a convex polygon with +2 sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).
- The number of ways to connect the 2n points on a circle to form n disjoint i.e. non-intersecting chords.
- The number of monotonic lattice paths from point (0,0) to point (n,n) in a square lattice of size $n \times n$, which do not pass above the main diagonal
- Number of permutation of length n that can be stack sorted.
- The number of non-crossing partitions of a set of n elements.
- The number of rooted full binary tree with n+1 leaves.
- The number of Dyck words of length 2n. A string consisting of n X's and n Y's such that no string prefix has more Y's than X's.
- Number of permutation of length n with no three-term increasing subsequence.
- Number of ways to tile a stairstep shape of height n with n rectangle.
- $C_n^k = \frac{k+1}{n+1}\binom{2n-k}{n-k}$ denote the number of bracket sequences of size 2n with the first k elements being (.
- $N(n,k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1}$
- The number of expressions containing n pairs of correct parentheses, which contain k distinct nestings. $N(4,2) = 6$
  $()((())), (())(()), (()(())), ((()())), ((())()), ((()))()$
- The number of paths from (0,0) to (2n, 0) with steps only northeast and southeast, not staying below the x-axis with k peaks. And sum of all number of peaks is Catalan number.

11.5.2 Stirling Numbers (2nd Kind)

$S(n,k) = kS(n-1,k) + S(n-1,k-1)$; partitions n objects into k non-empty subsets. See Stirling Numbers.cpp.

11.5.3 Lucas Theorem

$\binom{m}{n} \equiv \prod_{i=0}^{k} \binom{m_i}{n_i} \pmod{p}$ with base-p digits m_i, n_i. See Lucas Theorem.cpp.

11.5.4 Derangement

- A permutation such that no element appears in its original position.
- $d(n) = (n-1) * (d(n-1) + d(n-2)), d(0) = 1, d(1) = 0$
- $d(n) = nd(n-1) + (-1)^n = \lfloor \frac{n!}{e} \rfloor, n \geq 1$

11.6 Number Theory

11.6.1 Mobius Function and Inversion

$\mu(n) = (-1)^k$ for square-free n with k prime factors, 0 otherwise. $g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) g(n/d)$; $\sum_{d|n} \mu(d) = [n = 1]$. See Euler Phi and Mobius.cpp.

11.6.2 GCD and LCM

- gcd(a,b) = gcd(b, a mod b)
- If $a|b.c$, and gcd(a,b) = d, then $(a/d)|c$.
- GCD is a multiplicative function.
- gcd(a, lcm(b,c)) = lcm(gcd(a,b), gcd(a,c))
- $gcd(n^a - 1, n^b - 1) = n^{gcd(a,b)} - 1$

11.6.3 Pick's Theorem

According to Pick's Theorem We can calculate the area of any polygon by just counting the number of Interior and Boundary lattice points of that polygon. If number of interior points are I and number of boundary lattice points are B then Area (A) of polygon will be:

$$Area = I + B/2 - 1$$

where I is the number of points in the interior shape, B stands for the number of points on the boundary of the shape.

11.6.4 Formula Cheatsheet

- $\sum_{i=1}^{n} = \frac{1}{m+1}[(n+1)^{m+1} - 1 - \sum_{i=1}^{n}((i+1)^{m+1} - i^{m+1} - (m+1)i^m)]$
- $\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}, c \neq 1$
- $\sum_{i=0}^{\infty} c^i = \frac{1}{1-c}, \sum_{i=1}^{\infty} c^i = \frac{c}{1-c}, |c| < 1$
- $H_n = \sum_{i=1}^{n} \frac{1}{n}, \sum_{i=1}^{n} iH_i = \frac{n(n+1)}{2} H_n - \frac{n(n-1)}{4}$
- $\sum_{k=0}^{n} \binom{r+k}{k} = \binom{r+n+1}{n}$